UNIT 2

BASIC STRUCTURAL MODELLING

Object-oriented Modeling





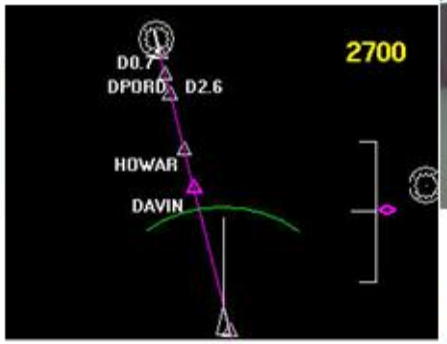




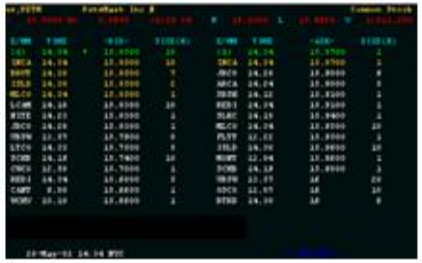




Application Domain
(Phenomena)







Solution Domain
(Phenomena)

System Model (Concepts) *(Analysis)*

UML Package

TrafficControl

Aircraft

TrafficController

Airport

FlightPlan

System Model (Concepts) *(Design)*

MapDisplay

Summary Display

FlightPlanDatabase

TrafficControl

UML Diagrams

- Types of UML Diagrams:
  - Use Case Diagram
  - Class Diagram
  - Sequence Diagram
  - Collaboration Diagram
  - State Diagram
  - Activity Diagram

Introduction

- Use case diagrams
  - Describe the **functional behavior** of the system as seen by the user
- Class diagrams
  - Describe the **static structure** of the system: **Objects, attributes, associations**
- Sequence diagrams
  - Describe the **dynamic behavior** between objects of the system
- State-chart diagrams
  - Describe the **dynamic behavior of an individual object**
- Activity diagrams
  - Describe the **dynamic behavior of a system**, i.e. **The Workflow.**

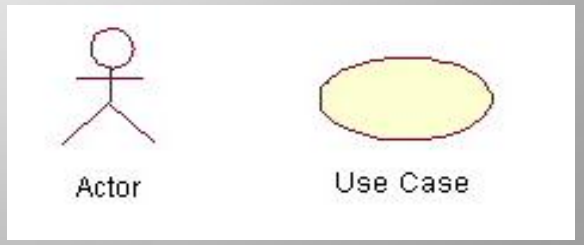
Use-Case Diagrams

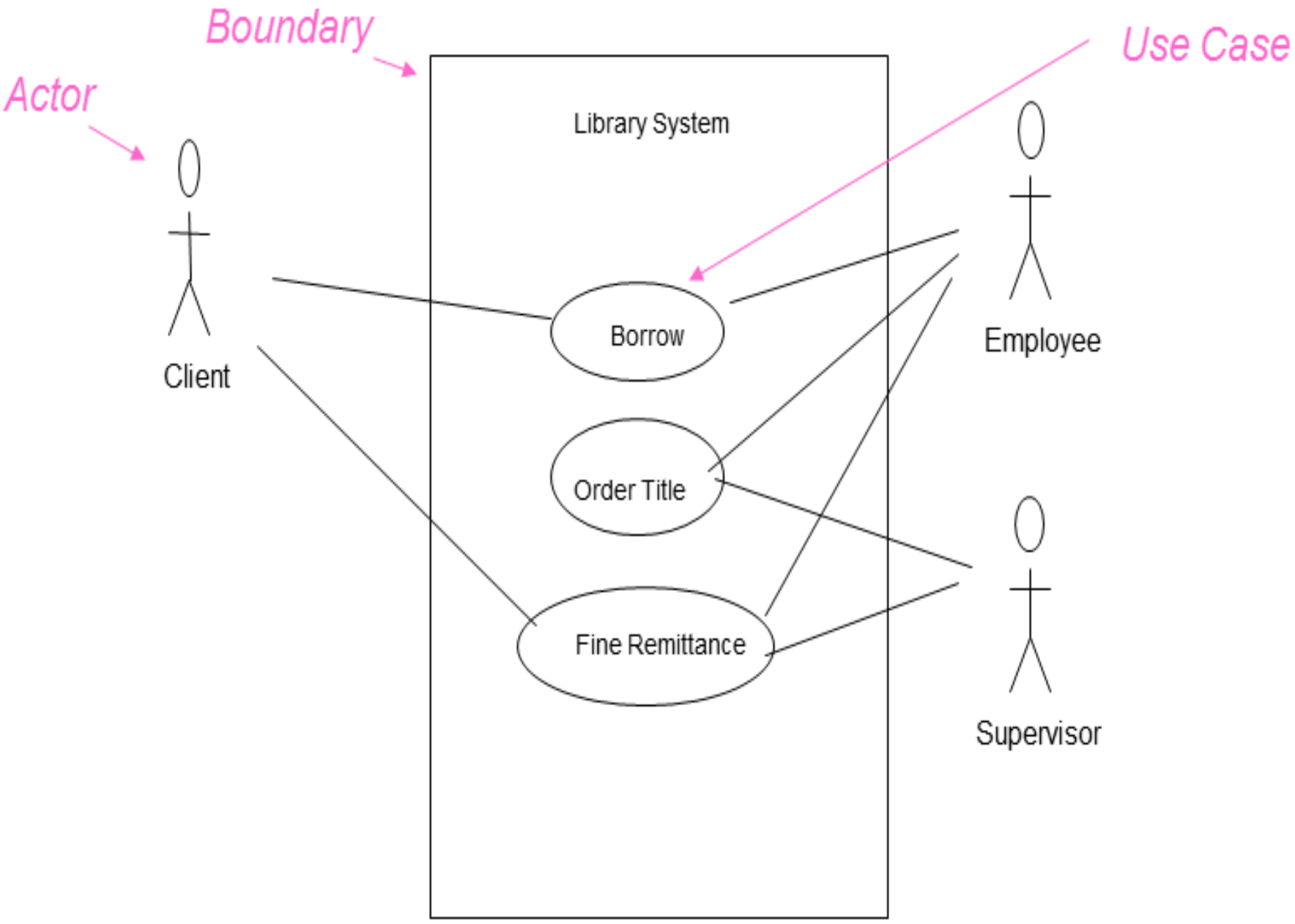
- A use-case diagram is a set of use cases
- A use case is a model of the interaction between
  - External users of a software product (actors) and
  - The software product itself
  - More precisely, an actor is a user playing a specific role

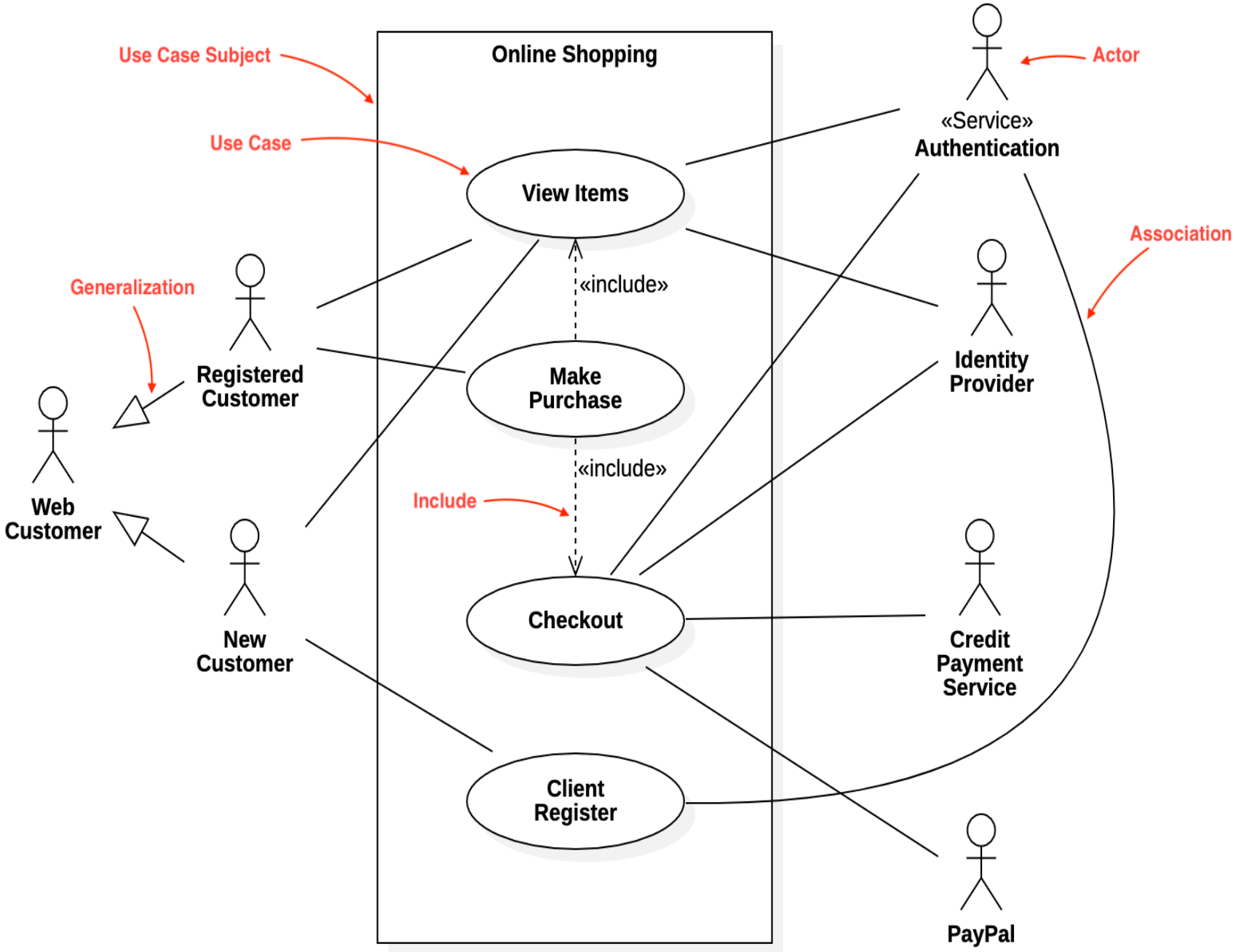
- describing a set of user **scenarios**
- capturing user requirements
- **contract** between end user and software developers

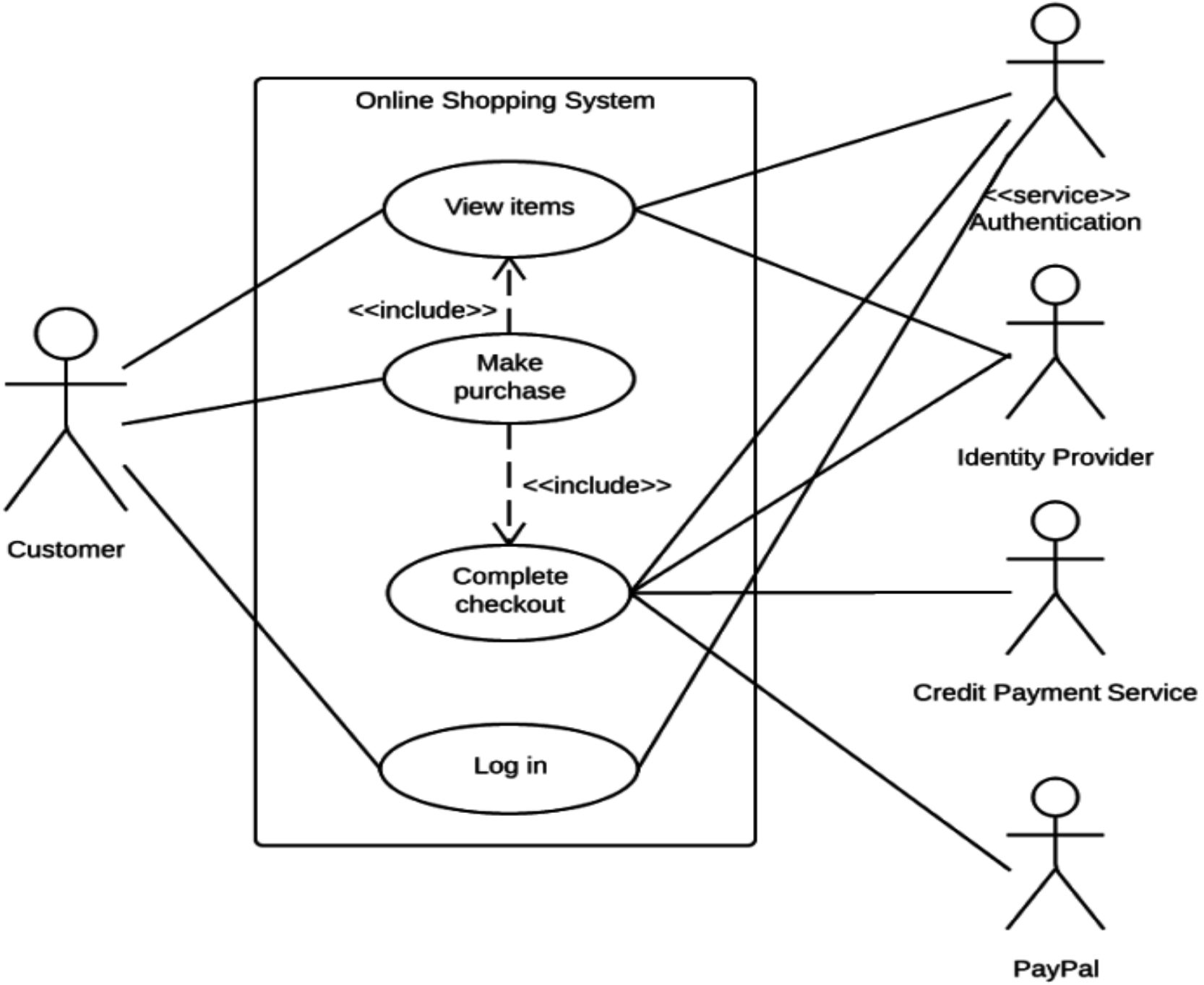
Use-Case Diagrams

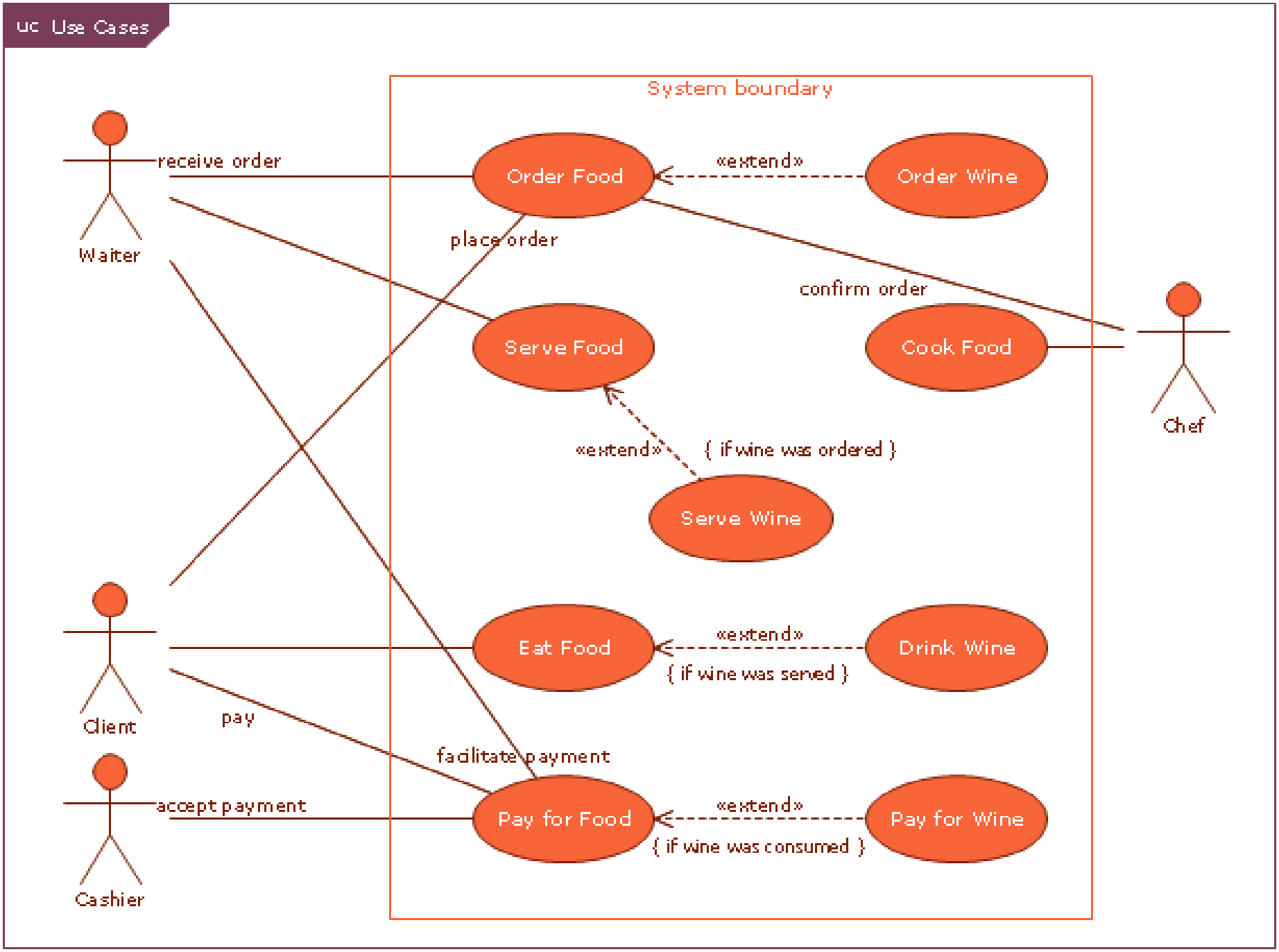
- **Actors:** A role that a user plays with respect to the system, including human users and other systems. e.g., inanimate physical objects (e.g. robot); an external system that needs some information from the current system.
- **Use case:** A set of scenarios that describing an interaction between a user and a system, including alternatives.
- **System boundary**: rectangle diagram representing the boundary between the actors and the system.



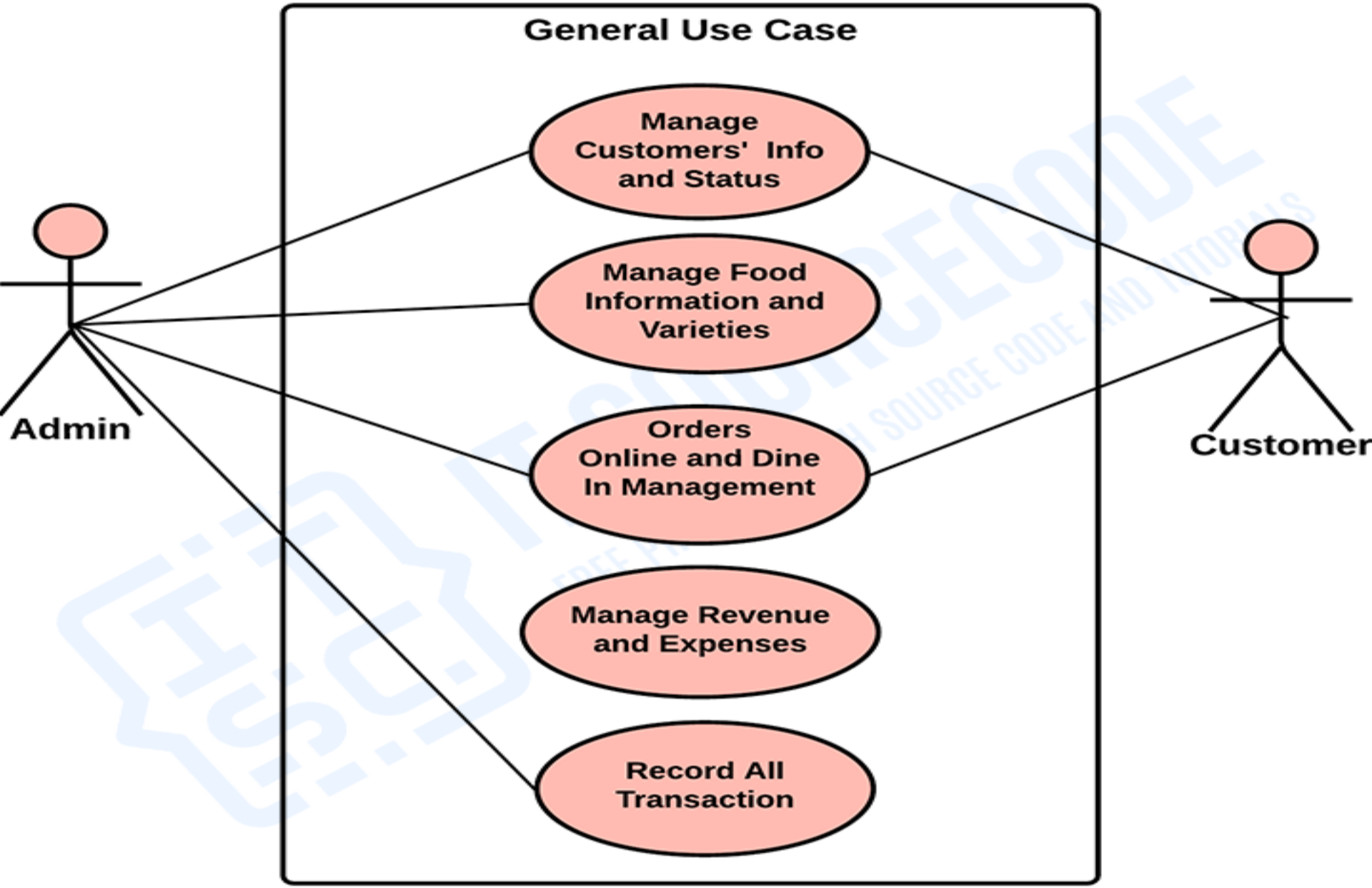

Boundary
Use Case
Actor
Library System
Client
Borrow
Employee
Order Title
Fine Remittance
Supervisor


Use Case Subject
Online Shopping
Actor
Use Case
«Service»
Authentication
View Items
Association
Generalization
«include»
Registered
Customer
Identity
Provider
Make
Purchase
«include»
Web
Customer
Include
Checkout
New
Customer
Credit
Payment
Service
Client
Register
PayPal


Online Shopping System
View items
<<include>>
Make purchase
<<include>>
Complete checkout
Log in
Customer
<<service>> Authentication
Identity Provider
Credit Payment Service
PayPal


uc Use Cases
System boundary
Waiter
receive order
Order Food
«extend»
Order Wine
place order
confirm order
Serve Food
Cook Food
Chef
«extend»
{ if wine was ordered }
Serve Wine
Client
pay
Eat Food
«extend»
Drink Wine
{ if wine was served }
facilitate payment
Cashier
accept payment
Pay for Food
«extend»
Pay for Wine
{ if wine was consumed }

RESTAURANT MANAGEMENT SYSTEM



USE CASE DIAGRAM

Class diagram

- A class diagram depicts classes and their interrelationships
- Used for describing structure and behavior in the use cases
- Provide a conceptual model of the system in terms of entities and their relationships
- Used for requirement capture, end-user interaction
- Detailed class diagrams are used for developers
- Depict the detailed design of object-oriented or object-based software

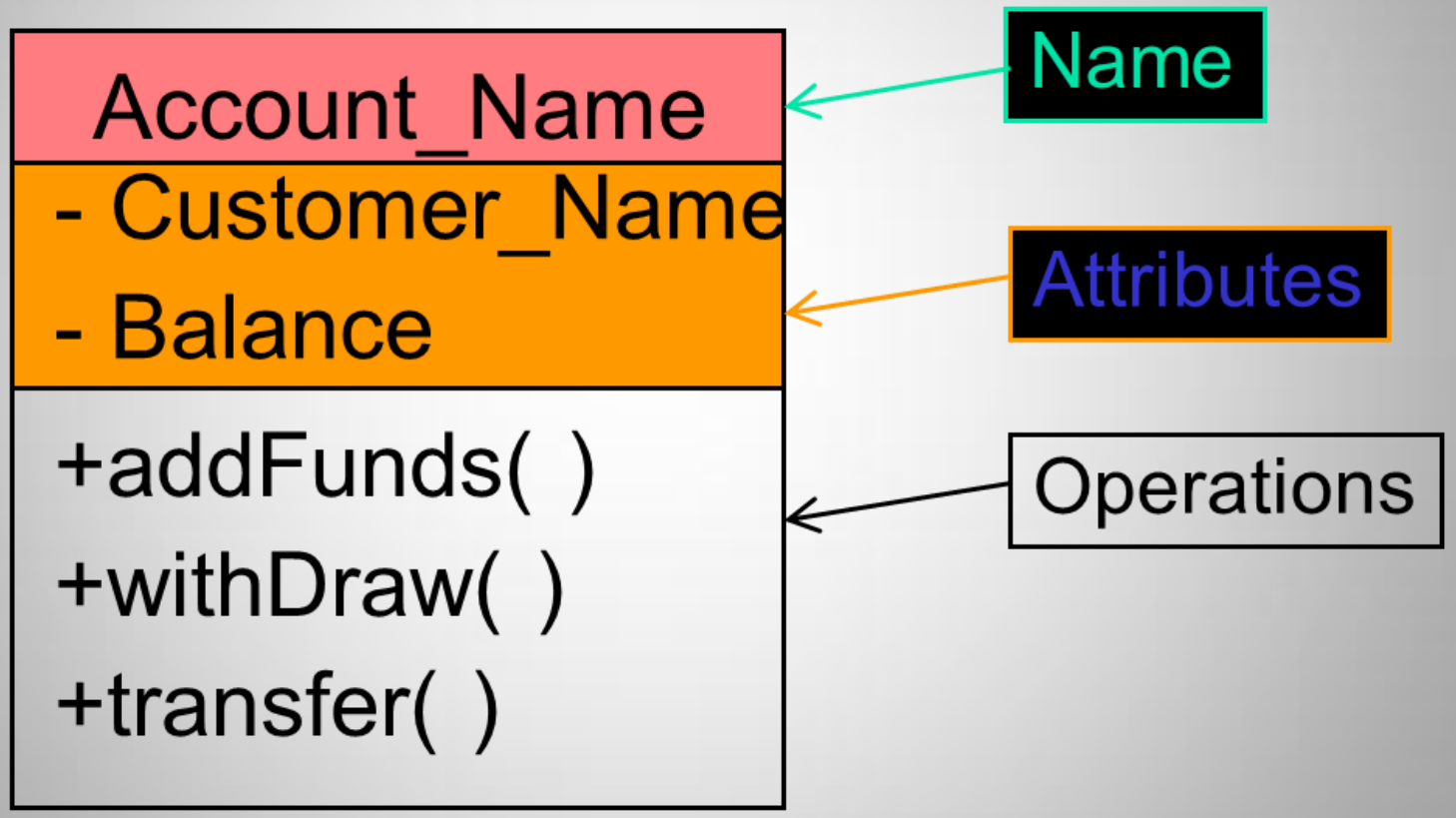
Class Diagram

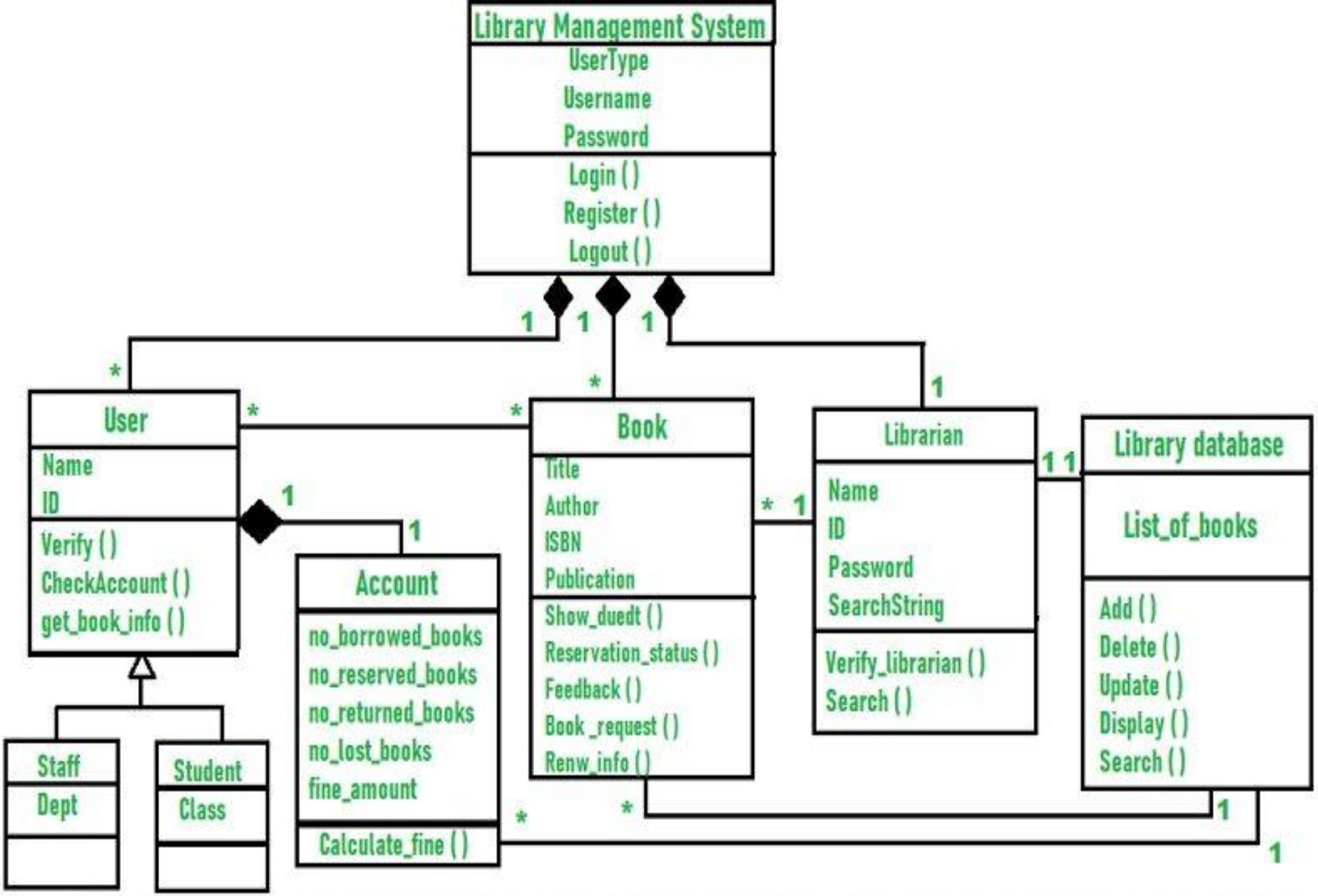
- Each class is represented by a rectangle subdivided into three compartments
  - Name
  - Attributes
  - Operations

- Modifiers are used to indicate visibility of attributes and operations.
- ‘+’ is used to denote Public visibility (everyone)
- ‘#’ is used to denote Protected visibility (friends and derived)
- ‘-’ is used to denote Private visibility (no one)

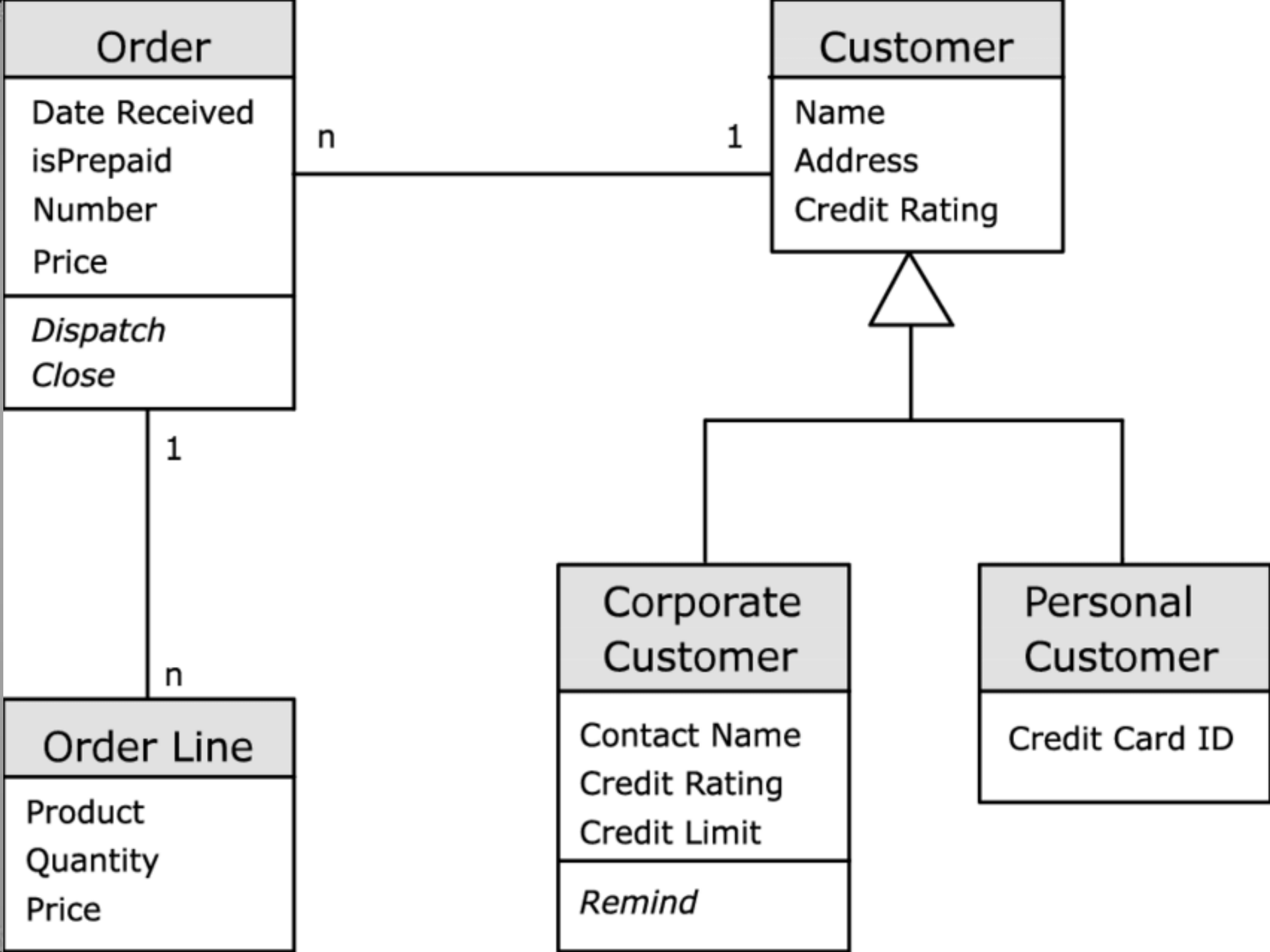
- By default, attributes are hidden and operations are visible.

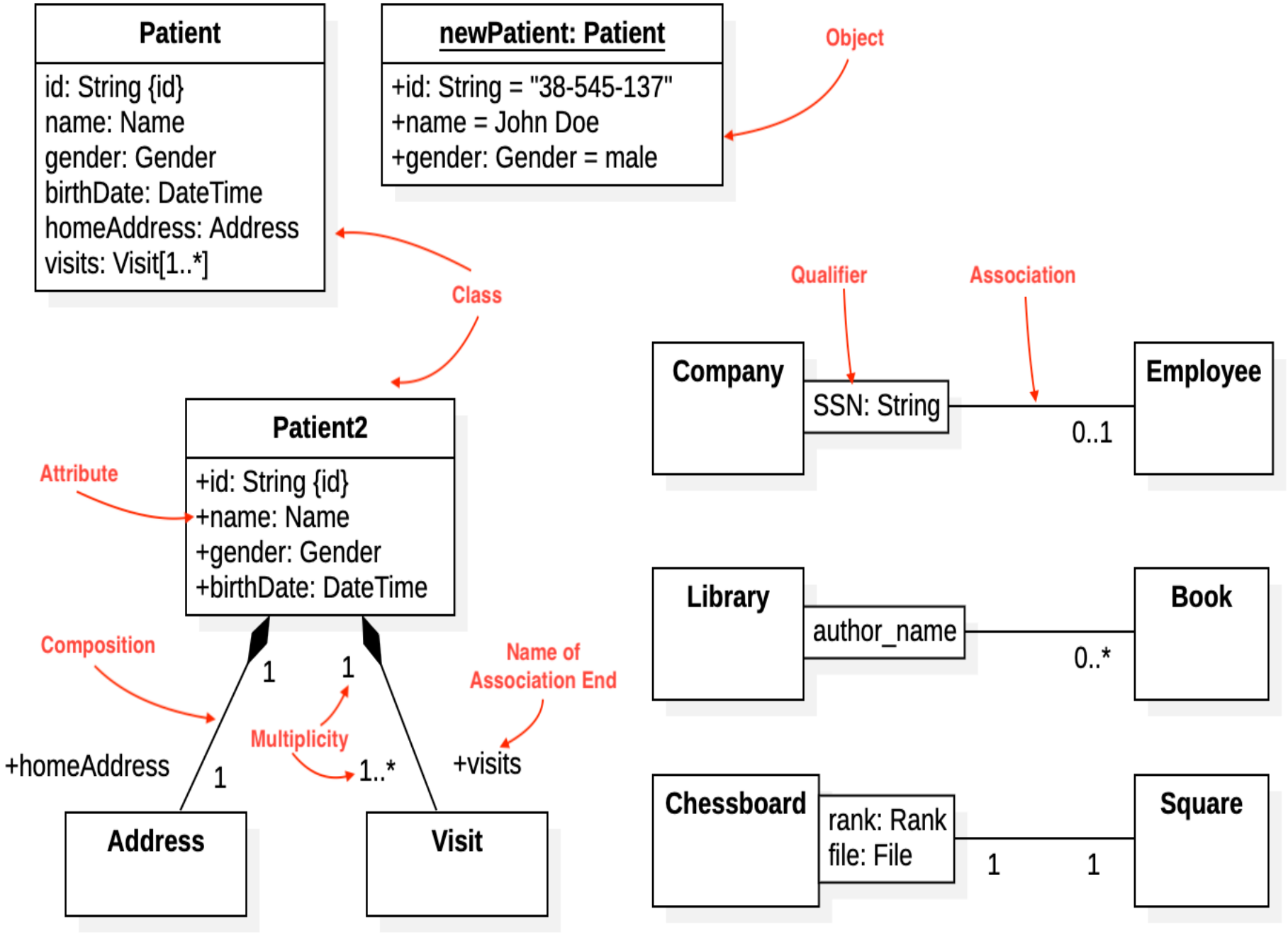
Class Diagram

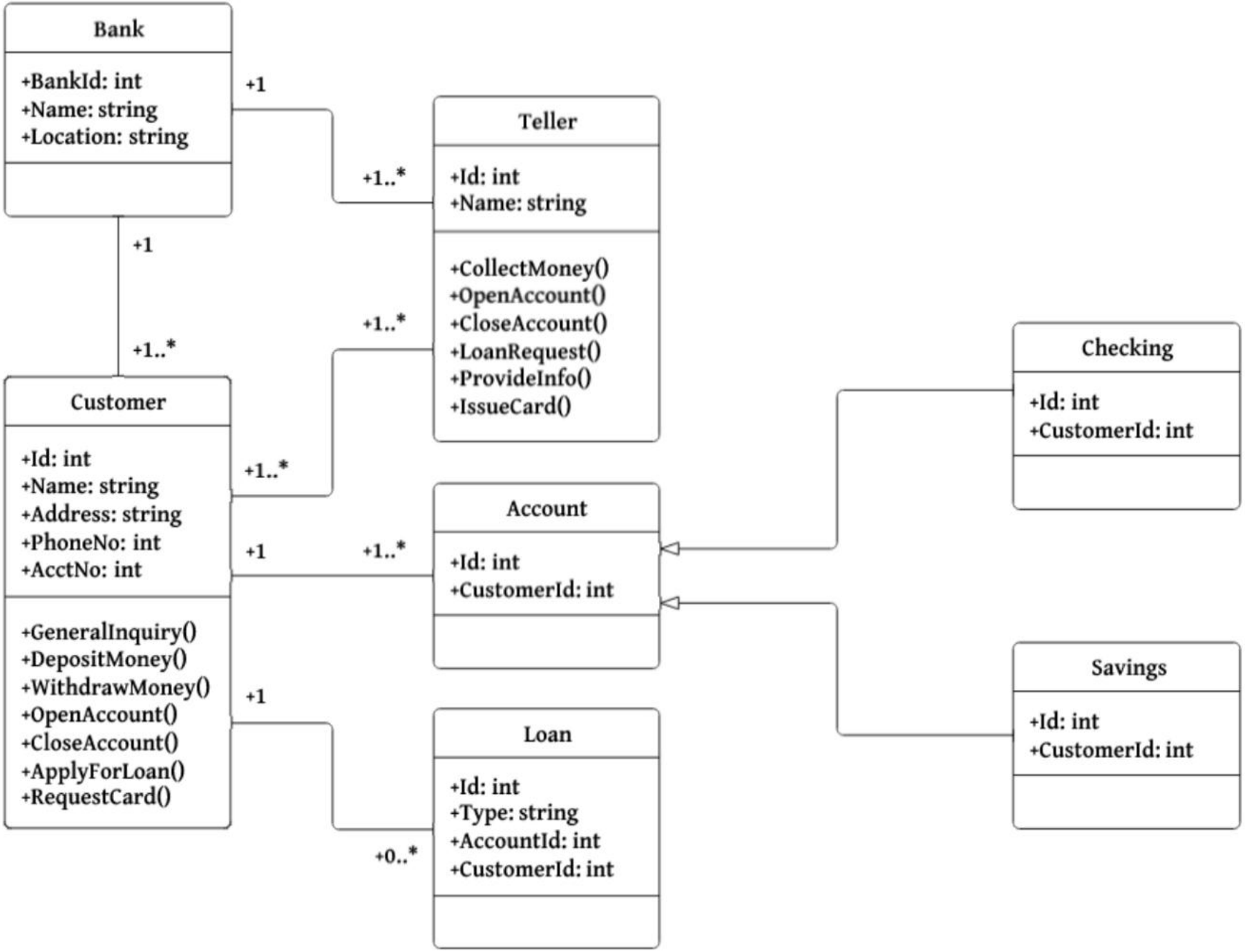




CLASS DIAGRAM FOR LIBRARY MANAGEMENT SYSTEM


Order
Date Received
isPrepaid
Number
Price
Dispatch
Close
n
1
Customer
Name
Address
Credit Rating
1
n
Order Line
Product
Quantity
Price
Corporate Customer
Contact Name
Credit Rating
Credit Limit
Remind
Personal Customer
Credit Card ID

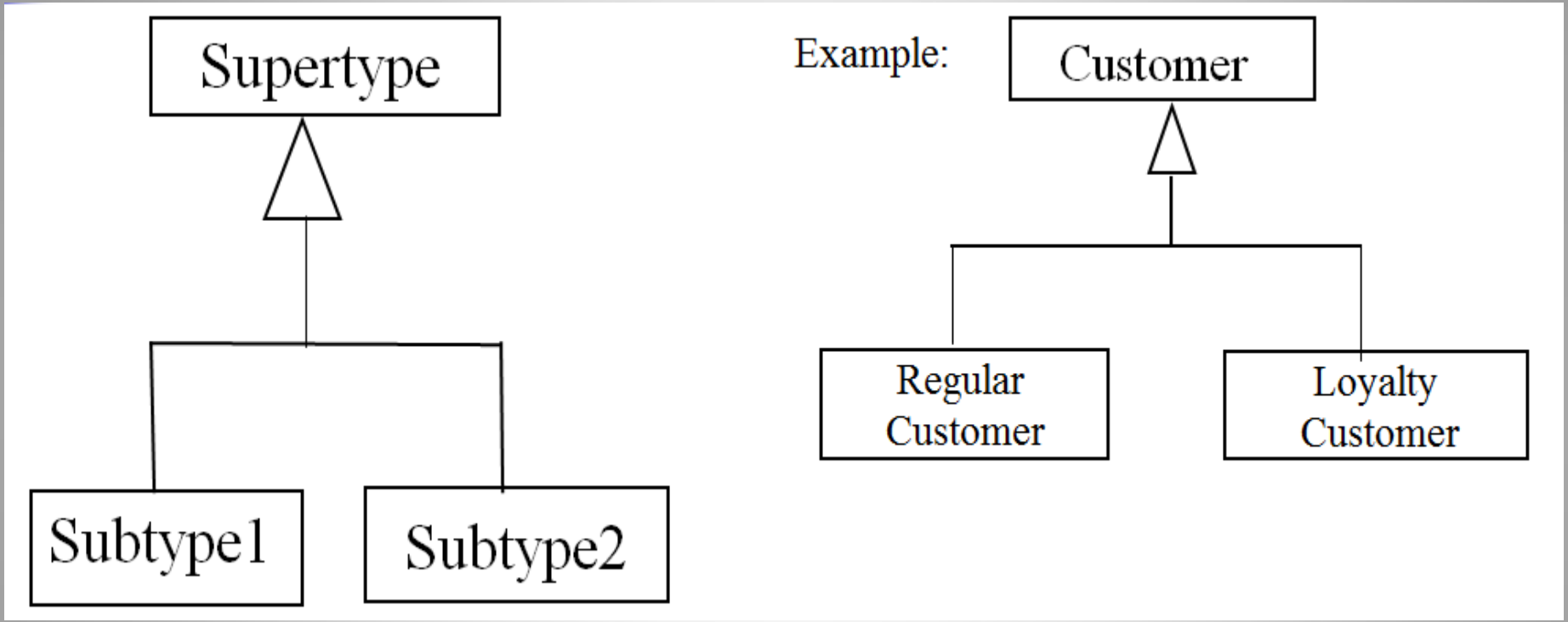

Patient
id: String {id}
name: Name
gender: Gender
birthDate: DateTime
homeAddress: Address
visits: Visit[1..*]
newPatient: Patient
+id: String = "38-545-137"
+name = John Doe
+gender: Gender = male
Object
Class
Patient2
+id: String {id}
+name: Name
+gender: Gender
+birthDate: DateTime
Attribute
Composition
1
1
Multiplicity
1..*
Name of
Association End
+homeAddress
1
+visits
Address
Visit
Qualifier
Association
Company
SSN: String
0..1
Employee
Library
author_name
0..*
Book
Chessboard
rank: Rank
file: File
1
1
Square


Bank
+BankId: int
+Name: string
+Location: string
+1
+1..*
Teller
+Id: int
+Name: string
+CollectMoney()
+OpenAccount()
+CloseAccount()
+LoanRequest()
+ProvideInfo()
+IssueCard()
+1
+1..*
+1..*
Customer
+Id: int
+Name: string
+Address: string
+PhoneNo: int
+AcctNo: int
+GeneralInquiry()
+DepositMoney()
+WithdrawMoney()
+OpenAccount()
+CloseAccount()
+ApplyForLoan()
+RequestCard()
+1..*
+1
+1..*
Account
+Id: int
+CustomerId: int
Checking
+Id: int
+CustomerId: int
Savings
+Id: int
+CustomerId: int
+1
+0..*
Loan
+Id: int
+Type: string
+AccountId: int
+CustomerId: int

OO Relationship

- There are two kinds of Relationships
  - Generalization (parent-child relationship)
  - Association (student enrolls in course)
- Associations can be further classified as
  - Aggregation
  - Composition

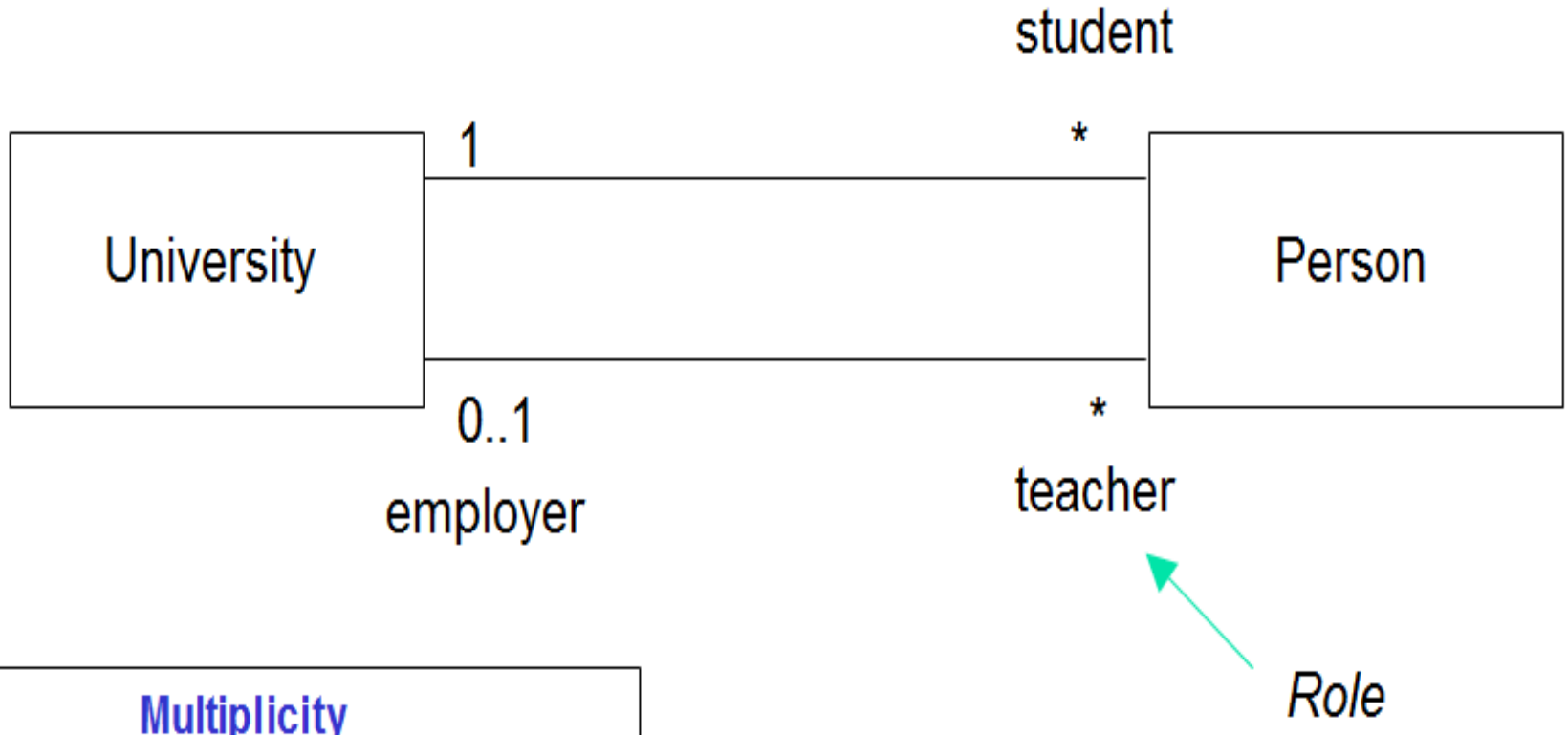
OO Relationships: **Generalization**



- Inheritance is a required feature of object orientation
- -Generalization expresses a parent/child relationship among related classes.
- Used for abstracting details in several layers

OO Relationships: **Association**

- Represent relationship between instances of classes
  - Student enrolls in a course
  - Courses have students
  - Courses have exams
  - Etc.
- Association has two ends
  - Role names (e.g. enrolls)
  - Multiplicity (e.g. One course can have many students)
  - Navigability (unidirectional, bidirectional)

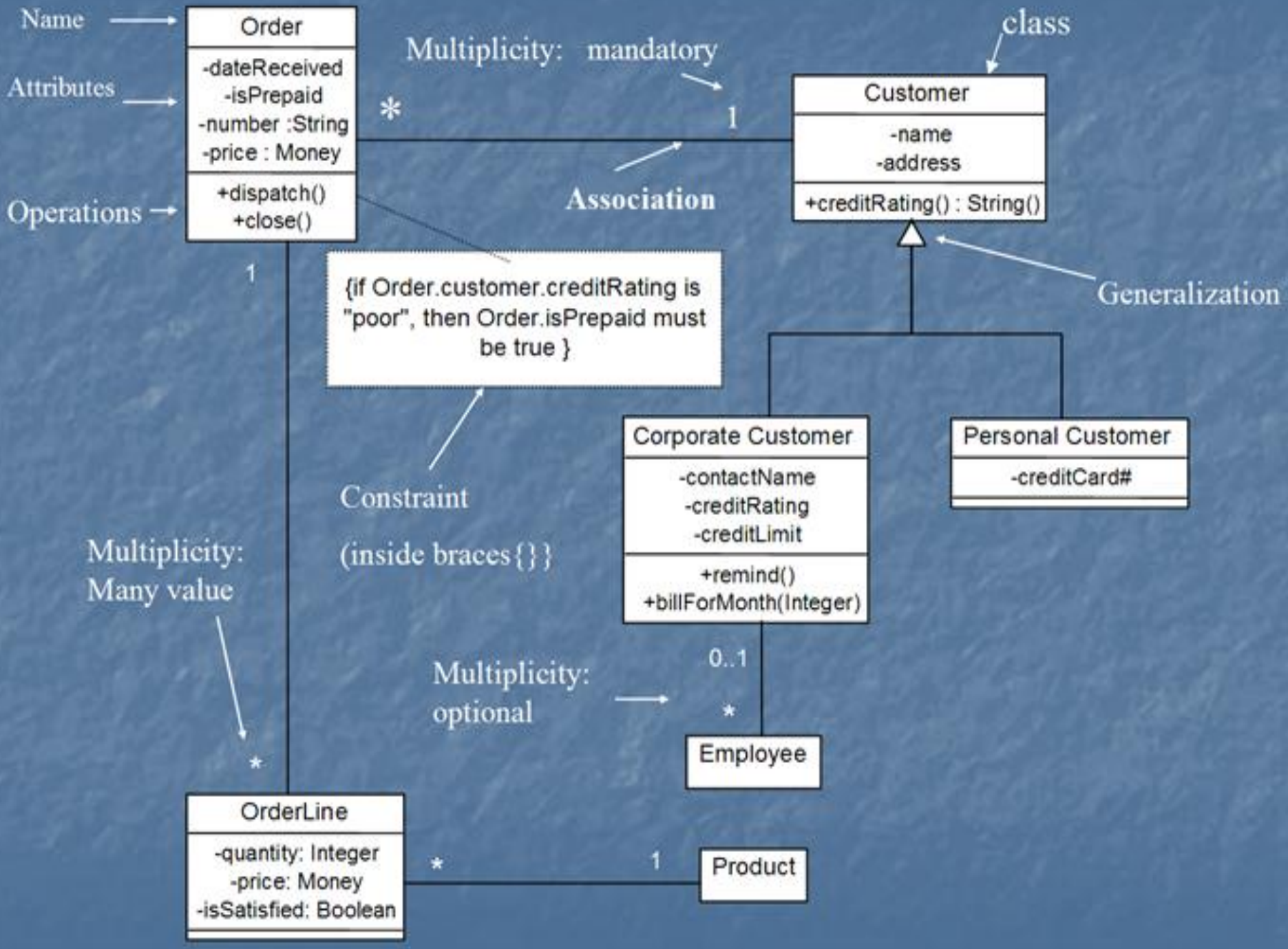


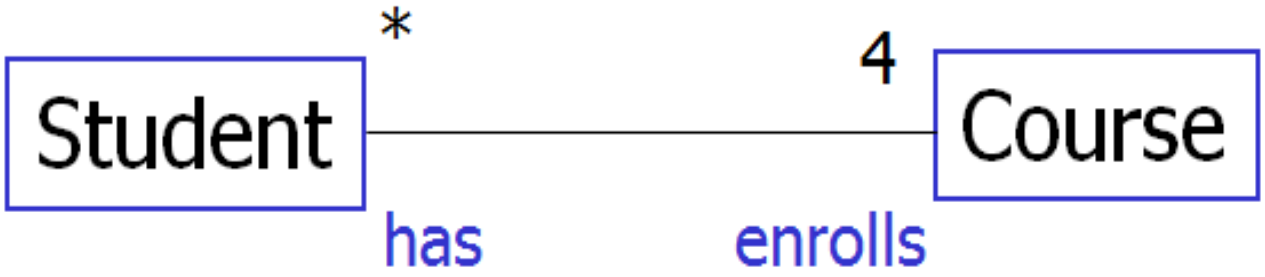
Multiplicity

Symbol	Meaning
1	One and only one
0..1	Zero or one
M..N	From M to N (natural language)
*	From zero to any positive integer
0..*	From zero to any positive integer
1..*	From one to any positive integer

Role

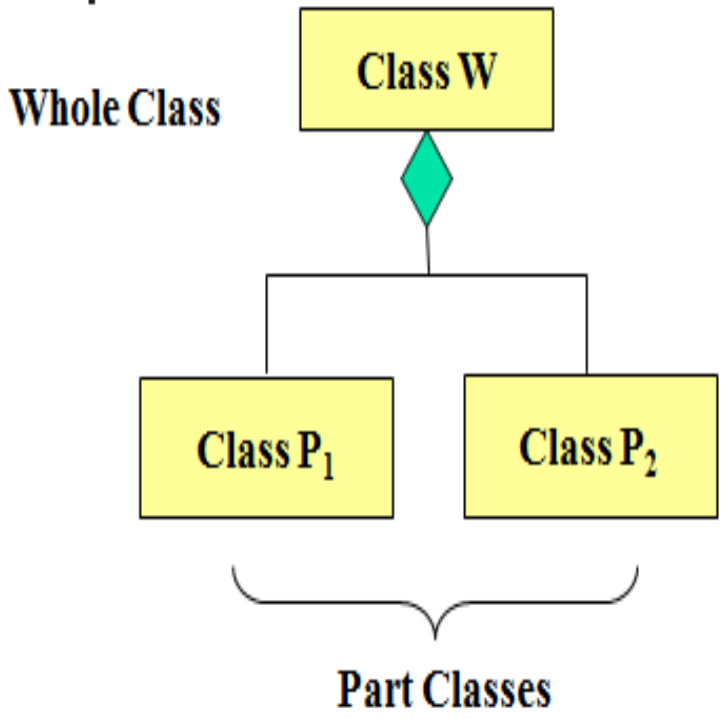
"A given university groups many people; some act as students, others as teachers. A given student belongs to a single university; a given teacher may or may not be working for the university at a particular time."


Name
Order
-dateReceived
-isPrepaid
-number :String
-price : Money
+dispatch()
+close()
Attributes
Operations
Multiplicity: mandatory
*
1
Association
class
Customer
-name
-address
+creditRating() : String()
Generalization
1
{if Order.customer.creditRating is "poor", then Order.isPrepaid must be true }
Constraint
(inside braces{})
Corporate Customer
-contactName
-creditRating
-creditLimit
+remind()
+billForMonth(Integer)
Personal Customer
-creditCard#
Multiplicity: Many value
*
Multiplicity: optional
0..1
*
Employee
OrderLine
-quantity: Integer
-price: Money
-isSatisfied: Boolean
*
1
Product




```
Class Student {
    Course enrolls[4];
}

Class Course {
    Student have[];
}
```



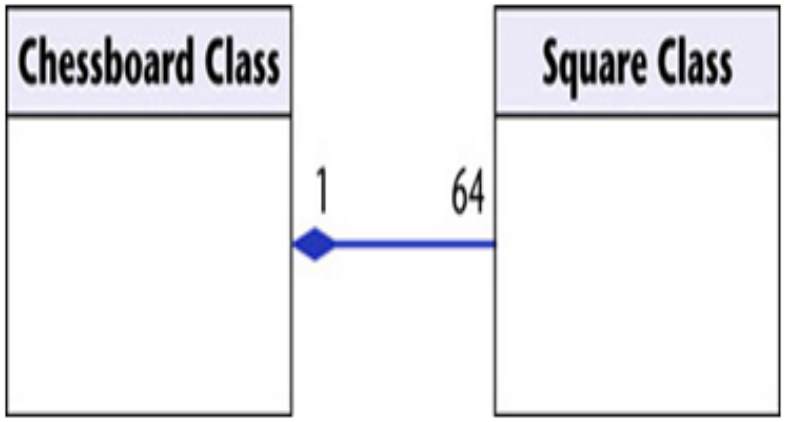
Association

Models the part–whole relationship

Composition

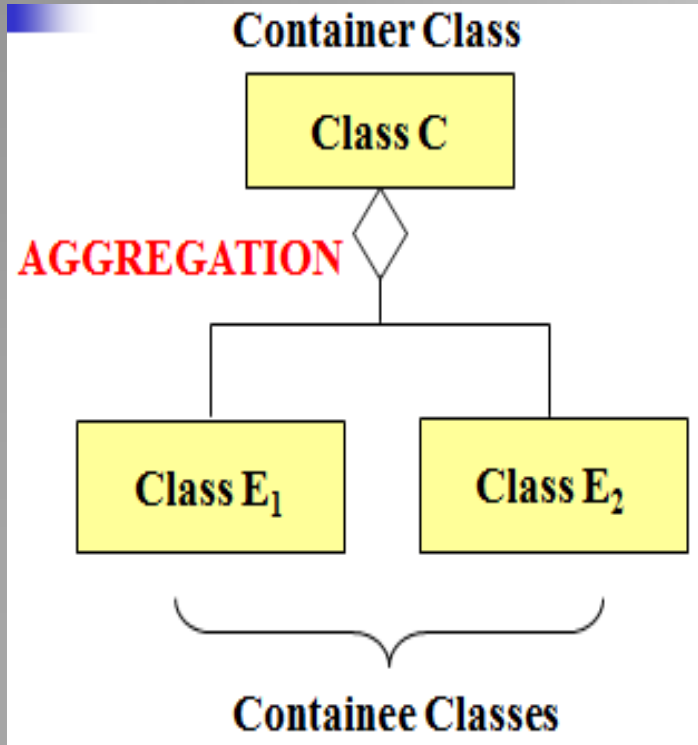
Also models the part–whole relationship but, in addition, Every part may belong to only one whole, and If the whole is deleted, so are the parts

Example



Example:

A number of different chess boards: Each square belongs to only one board. If a chess board is thrown away, all 64 squares on that board go as well.

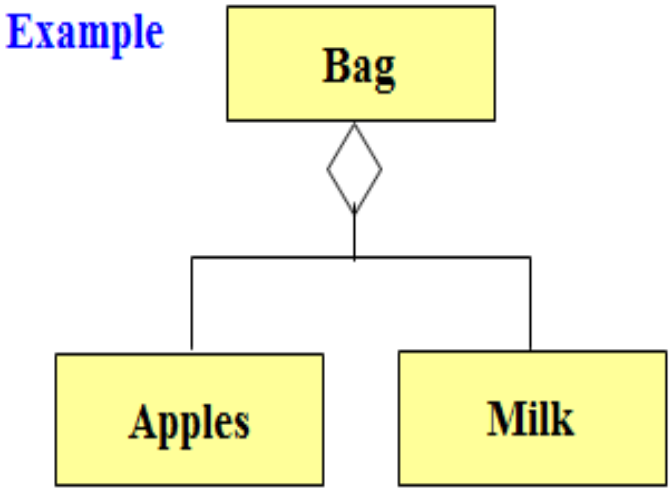


Aggregation:

expresses a relationship among instances of related classes. It is a specific kind of Container-Containee relationship.

express a more informal relationship than composition expresses.

Aggregation is appropriate when Container and Containees have no special access privileges to each other.



Aggregation vs. Composition

- **Composition** is really a strong form of **association**
  - components have only one owner
  - components cannot exist independent of their owner
  - components live or die with their owner
  - e.g. Each car has an engine that can not be shared with other cars.

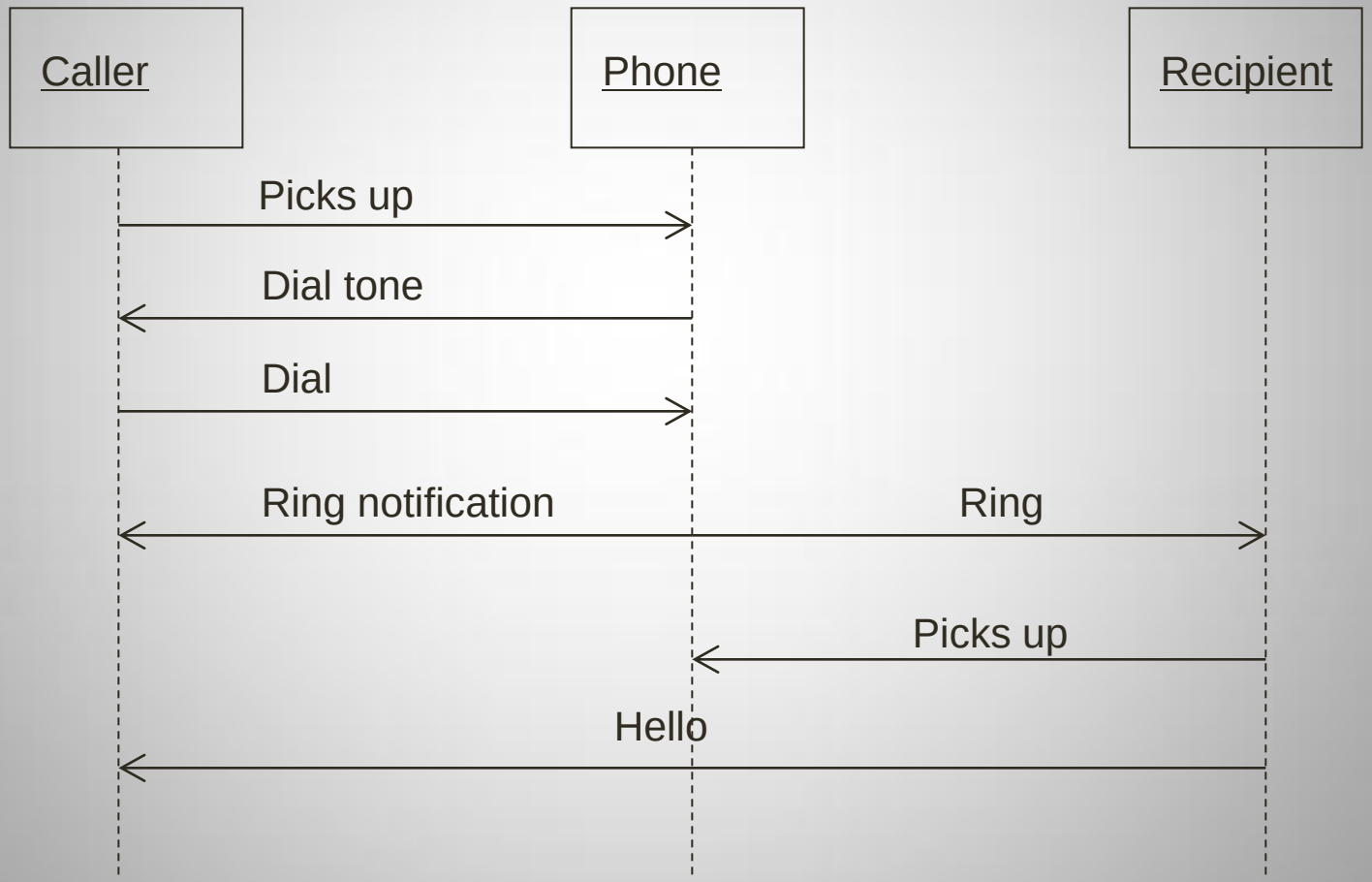
- **Aggregations**
  - may form "part of" the association, but may not be essential to it. They may also exist independent of the aggregate. e.g. Apples may exist independent of the bag.

Interaction Diagrams

- show how objects interact with one another

- UML supports two types of interaction diagrams
  - Sequence diagrams
  - Collaboration diagrams

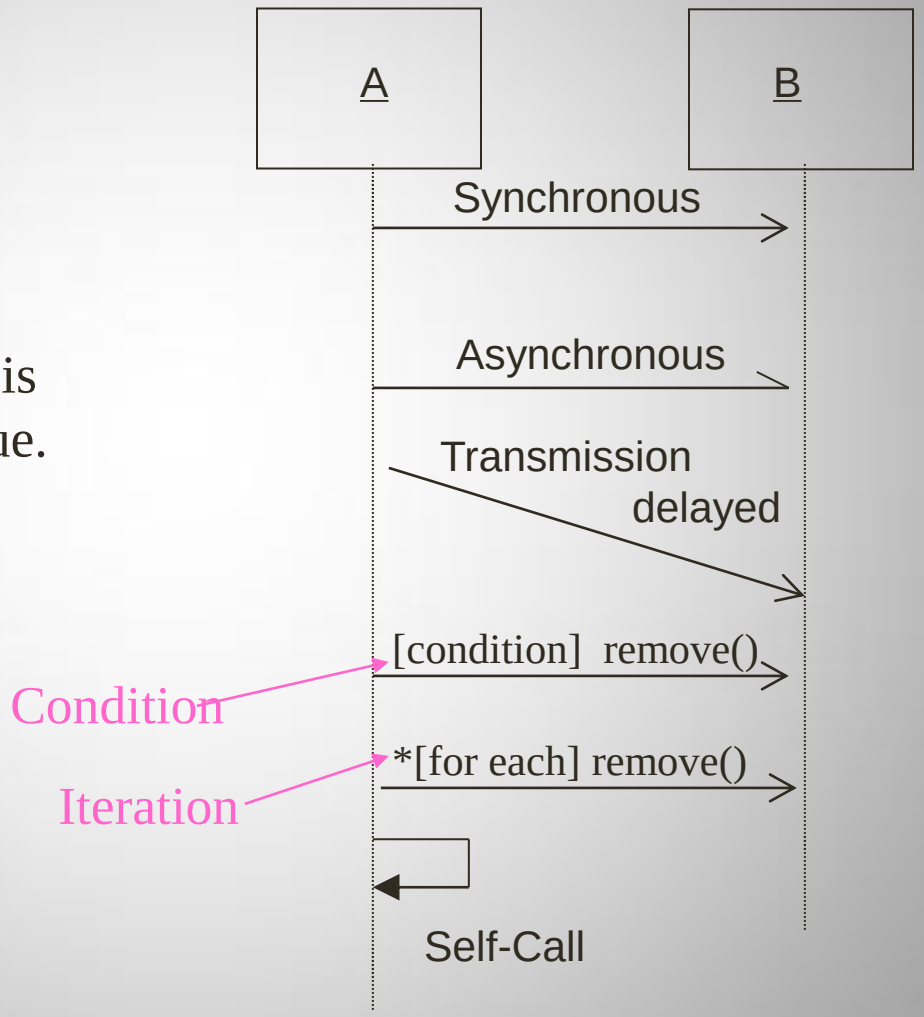
Sequence Diagram(make a phone call)



Sequence Diagram:Object interaction

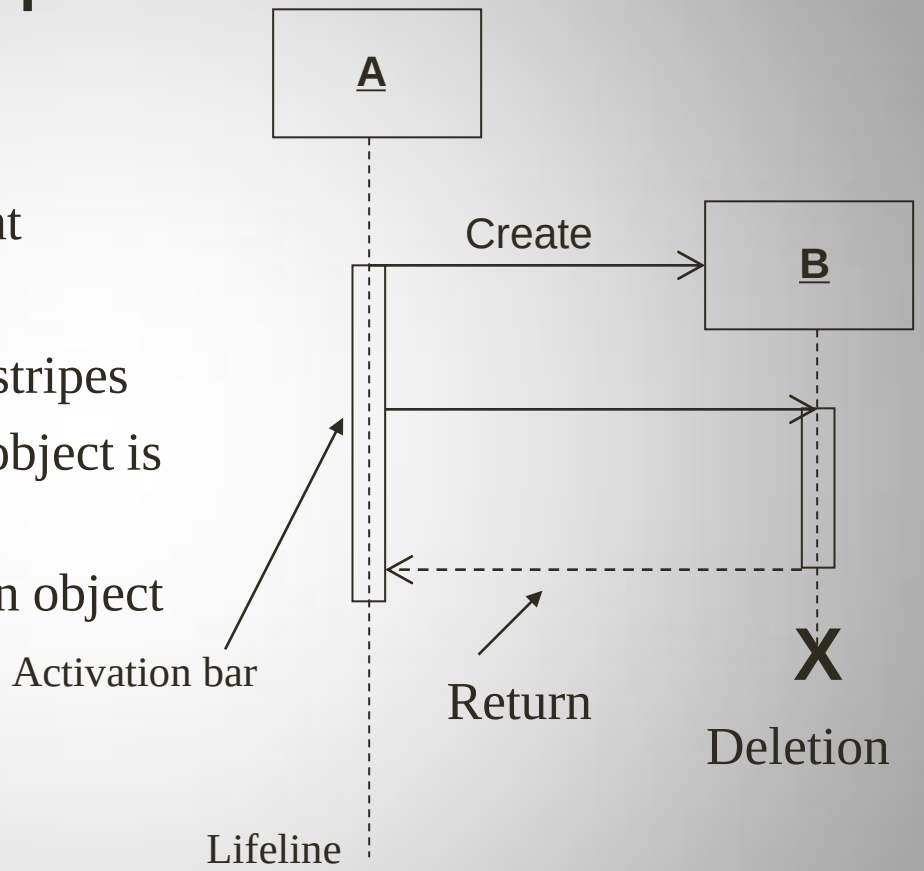
Self-Call: A message that an Object sends to itself.

Condition: indicates when a message is sent. The message is sent only if the condition is true.



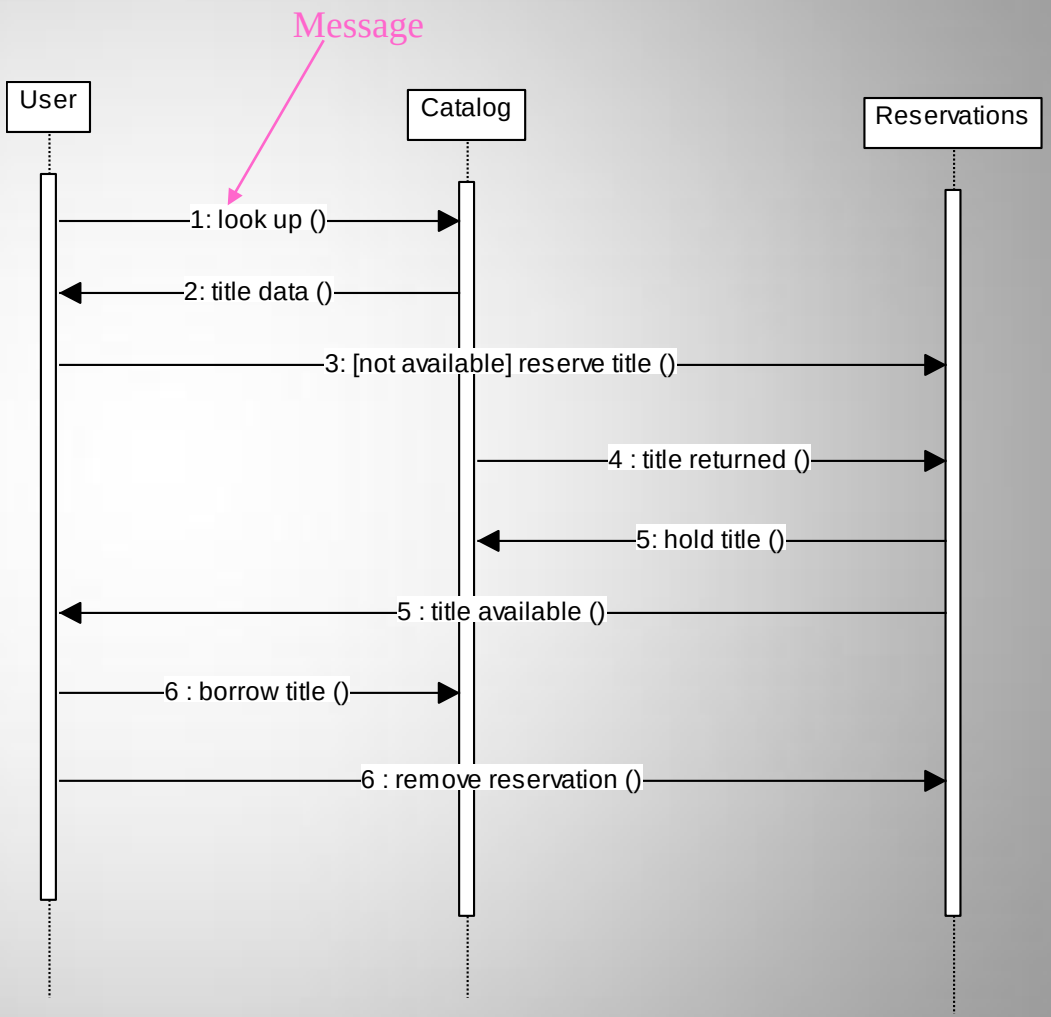
Sequence Diagrams – Object Life Spans

- Creation
  - Create message
  - Object life starts at that point
- Activation
  - Symbolized by rectangular stripes
  - Place on the lifeline where object is activated.
  - Rectangle also denotes when object is deactivated.
- Deletion
  - Placing an ‘X’ on lifeline
  - Object’s life ends at that point



Sequence Diagram

- Sequence diagrams demonstrate the behavior of objects in a use case by describing the objects and the messages they pass.
- The horizontal dimension shows the objects participating in the interaction.
- The vertical arrangement of messages indicates their order.
- The labels may contain the seq. # to indicate concurrency.



What is a Collaboration Diagram

- Collaboration diagrams illustrate interactions between objects
- The collaboration diagram illustrates messages being sent between classes and objects (instances).
- Collaboration diagrams express both the context of a group of objects (through objects and links) and the interaction between these objects (by representing message broadcasts)

Purpose of the Collaboration Diagram

- They are very useful for visualizing the relationship between objects collaborating to perform a particular task
- They provide a good view – albeit static - view of interaction between objects which may be difficult to see at the class level

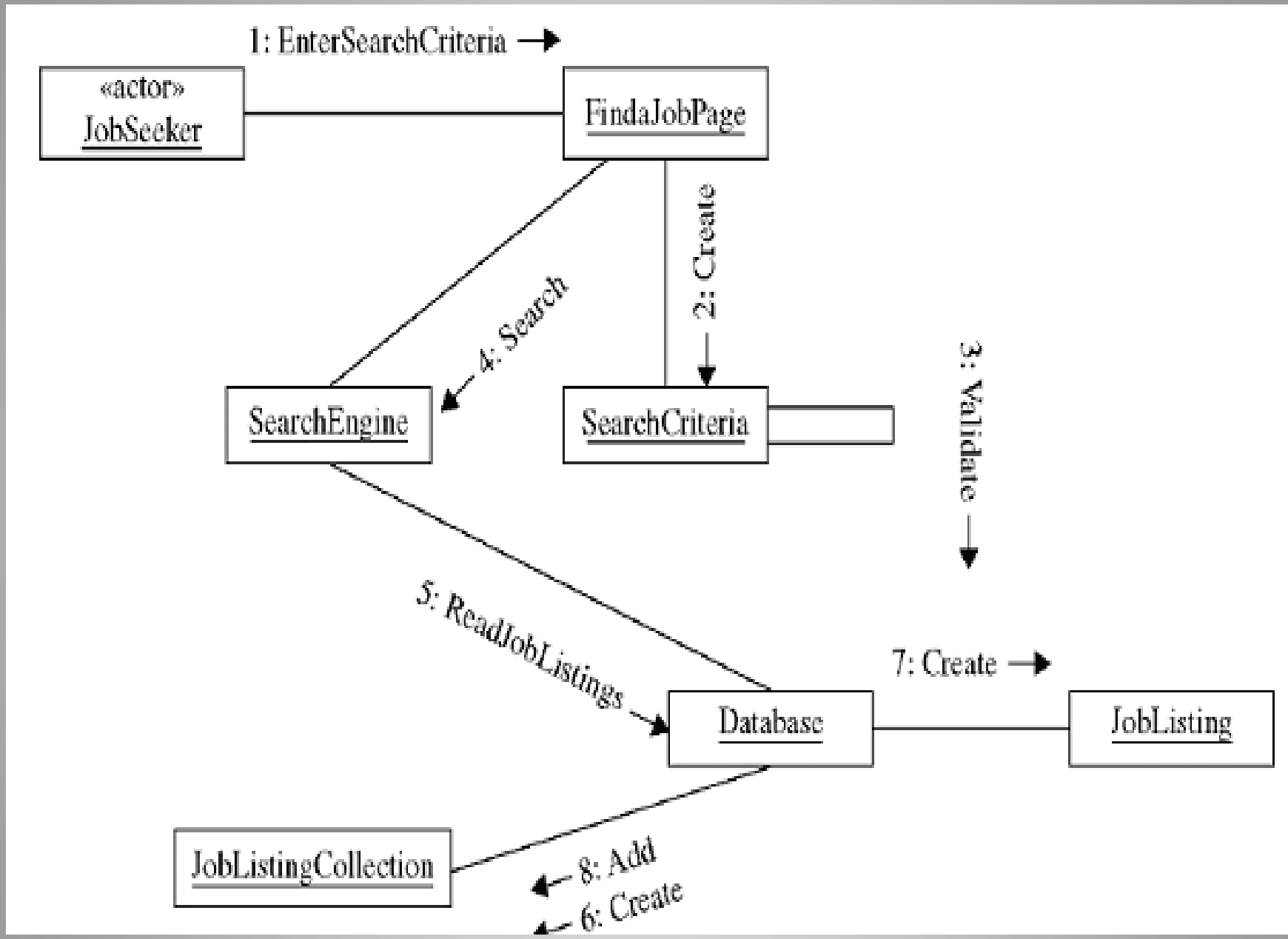
Collaboration Diagram

- Represents a Collaboration and Interaction
- Collaborationset of objects and their interactions in a specific context
- Interactionset of messages exchanged in a collaboration to produce a desired result

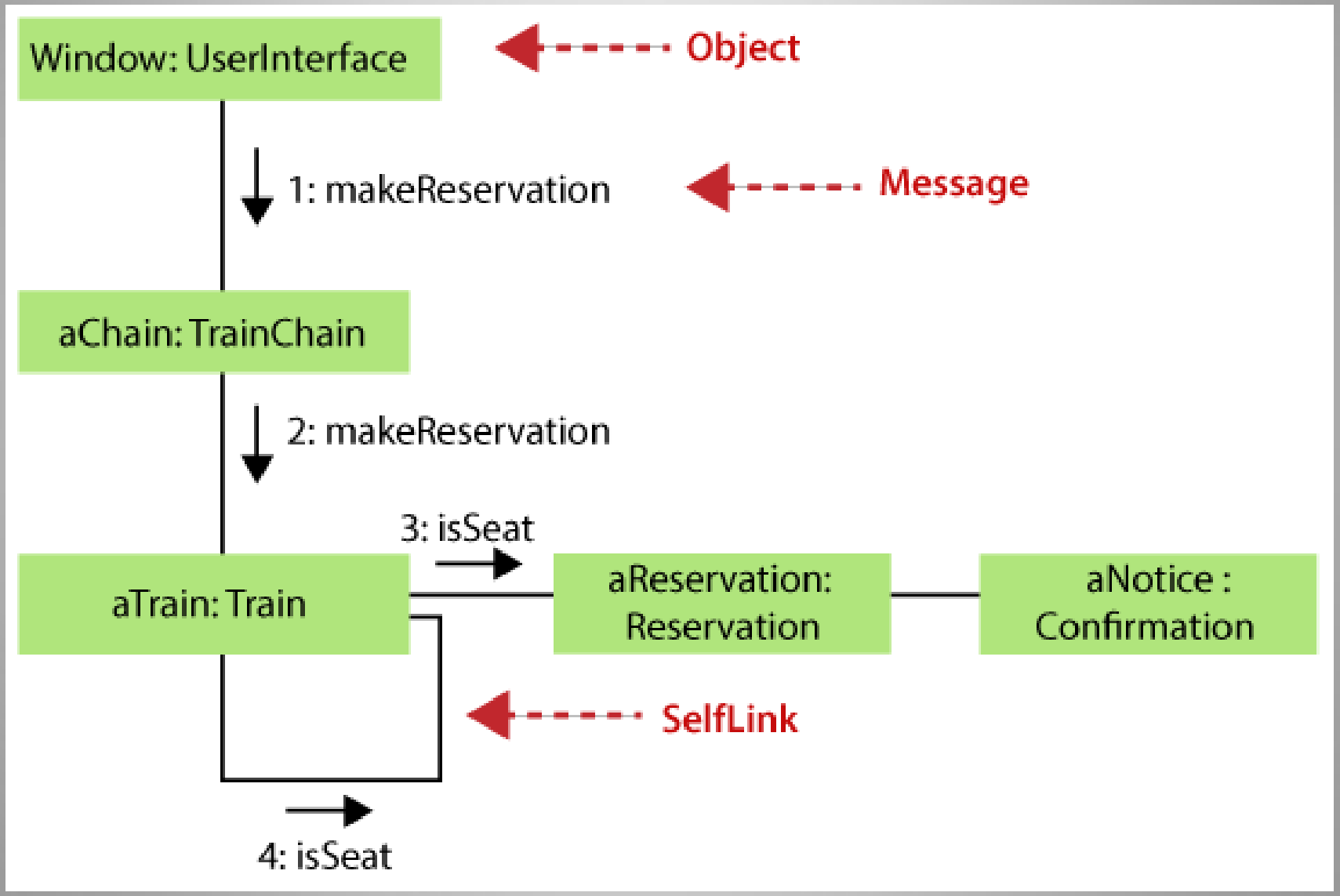
Components

- **Objects**
- **Actor**
- **Links**
- **Messages**

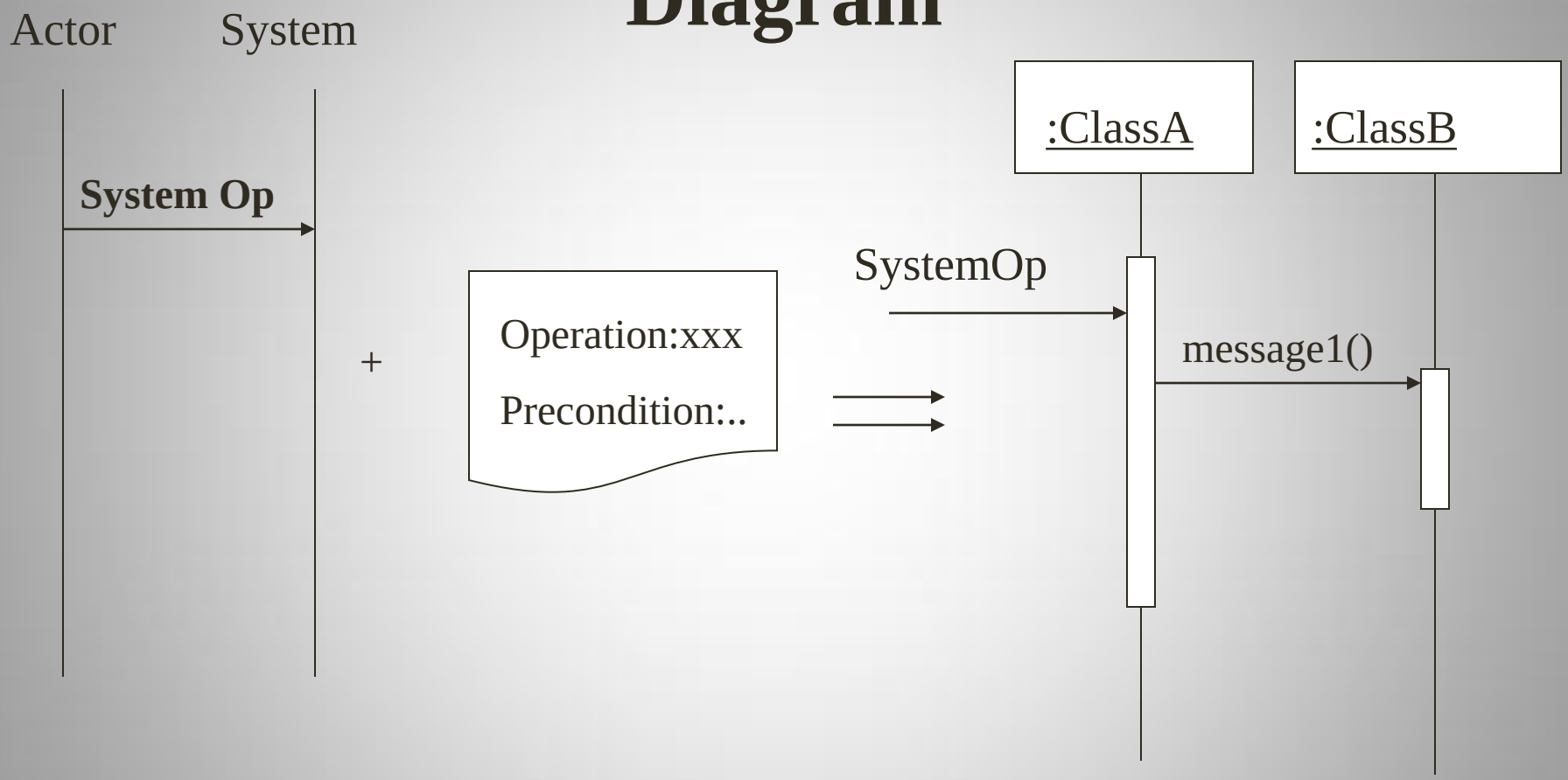
- **Also known as Communication Diagrams**


1: EnterSearchCriteria
«actor»
JobSeeker
FindaJobPage
2: Create
4: Search
SearchEngine
SearchCriteria
3: Validate
5: ReadJobListings
7: Create
Database
JobListing
JobListingCollection
8: Add
6: Create

Ex. Collaboration Diagram



Sequence Diagram VS Collaboration Diagram



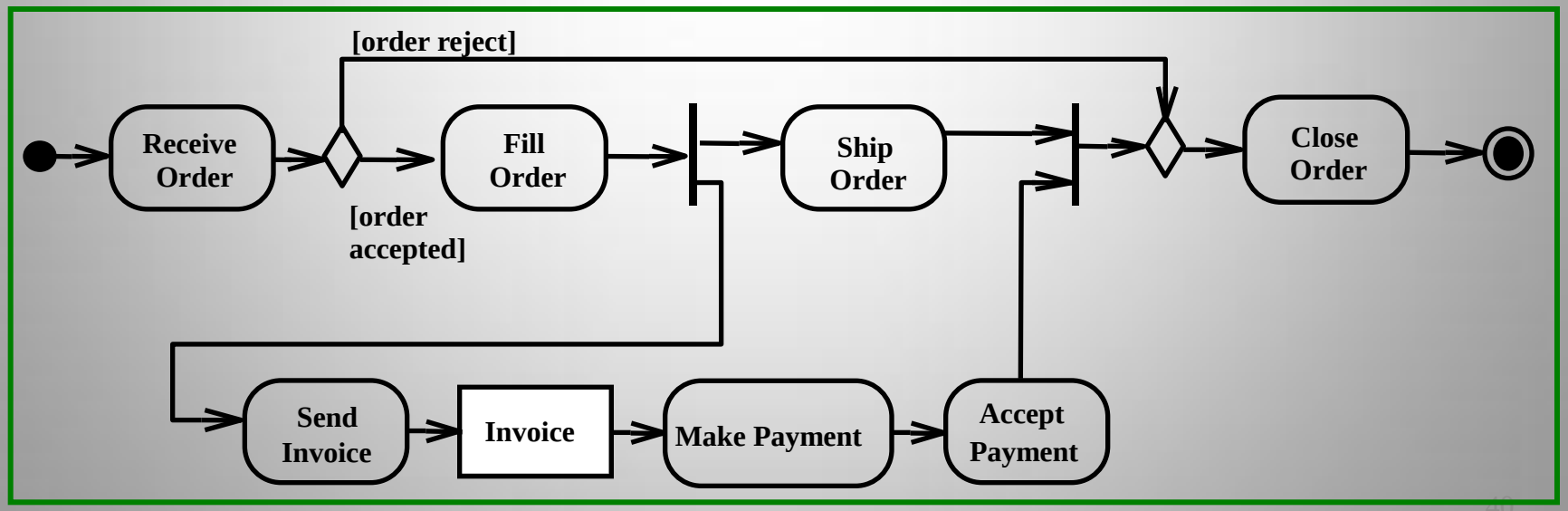
Sequence Diagram

Contract

Collaboration Diagram

Activity diagrams

- Useful to specify software or hardware system behaviour
- Based on data flow models – a graphical representation (with a Directed Graph) of how data move around an information system

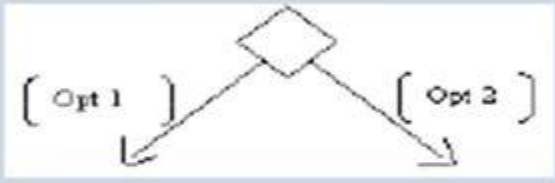
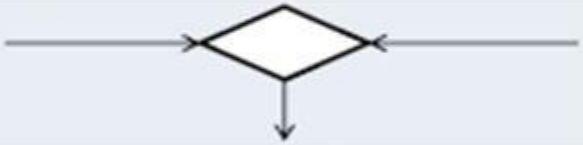
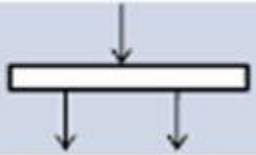
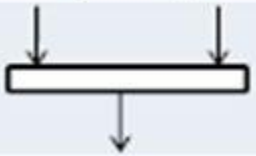


Some definitions

- ***Flow***: permits the interaction between two nodes of the activity diagram (represented by edges in activity diagram)
- ***State***: a condition or situation in the life of an object during which it satisfies some conditions, performs some activities, or waits for some events
- ***Type***: specifies a domain of objects together with the operations applicable to the objects (also none); includes primitive built-in types (such as integer and string) and enumeration types
- ***Token***: contains an object, datum, or locus of control, and is present in the activity diagram at a particular node; each token is distinct from any other, even if it contains the same value as another
- ***Value***: an element of a type domain

UML : ACTIVITY DIAGRAM

- Elements of activity diagram

Description	symbol
A Decision Node: Is used to represent a test condition to ensure that the control flow or object flow only goes down one path	
A Merge Node: Is used to bring back together different decision paths that were created using a decision-node.	
A Fork Node:Is used to split behavior into a set of parallel or concurrent flows of activities (or actions)	
A Join Node: Is used to bring back together a set of parallel or concurrent flows of activities (or actions).	
A Swimlane :A swimlane is a way to group activities performed by the same actor on an activity diagram or to group activities in a single thread	

Activity Diagram

Action
A step in the flow of events

Decision
Flows split based on a guard condition

Fork
Beginning of concurrent flows

Join
End of concurrent flow

Flow
Show the sequence of activities

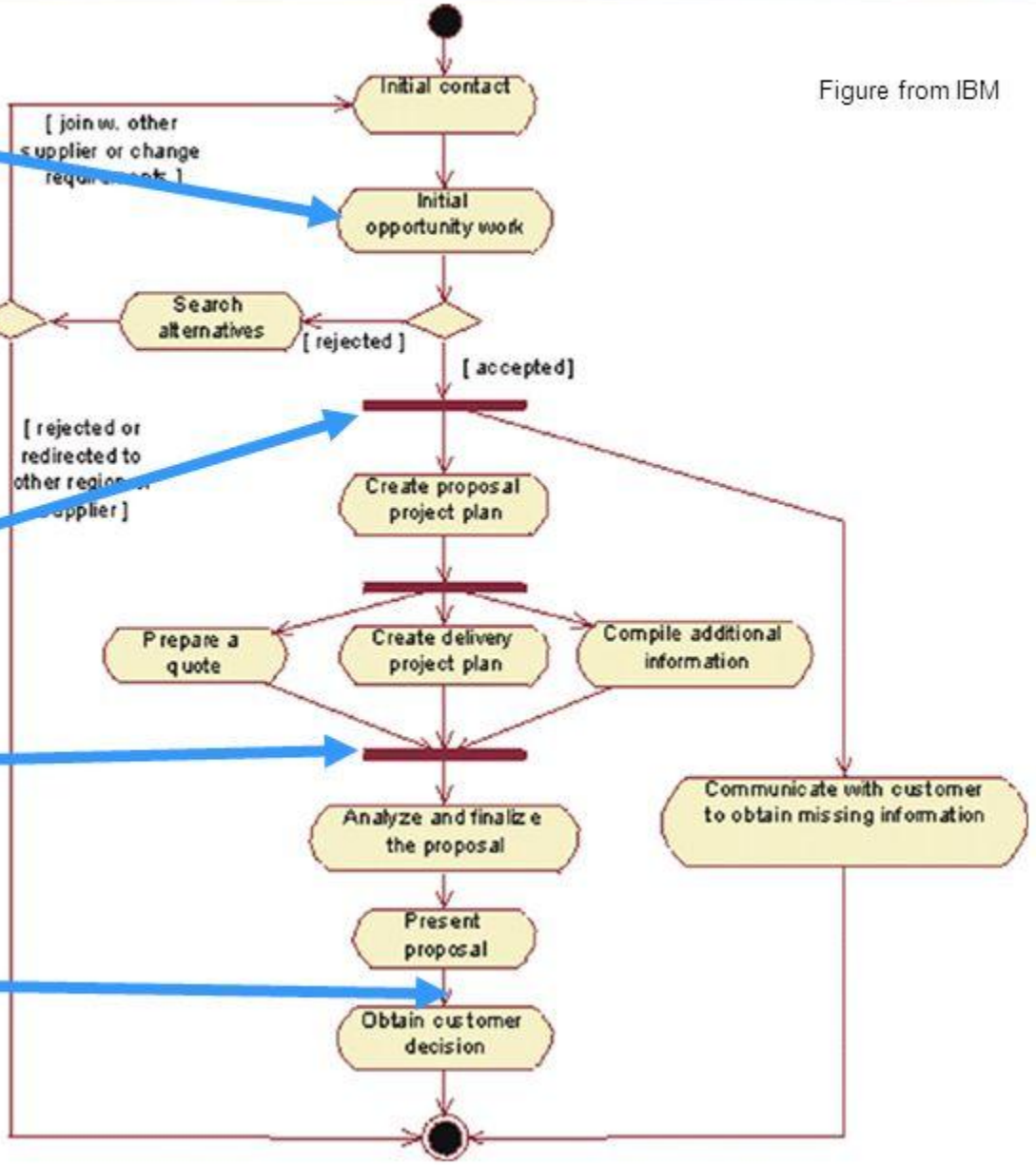
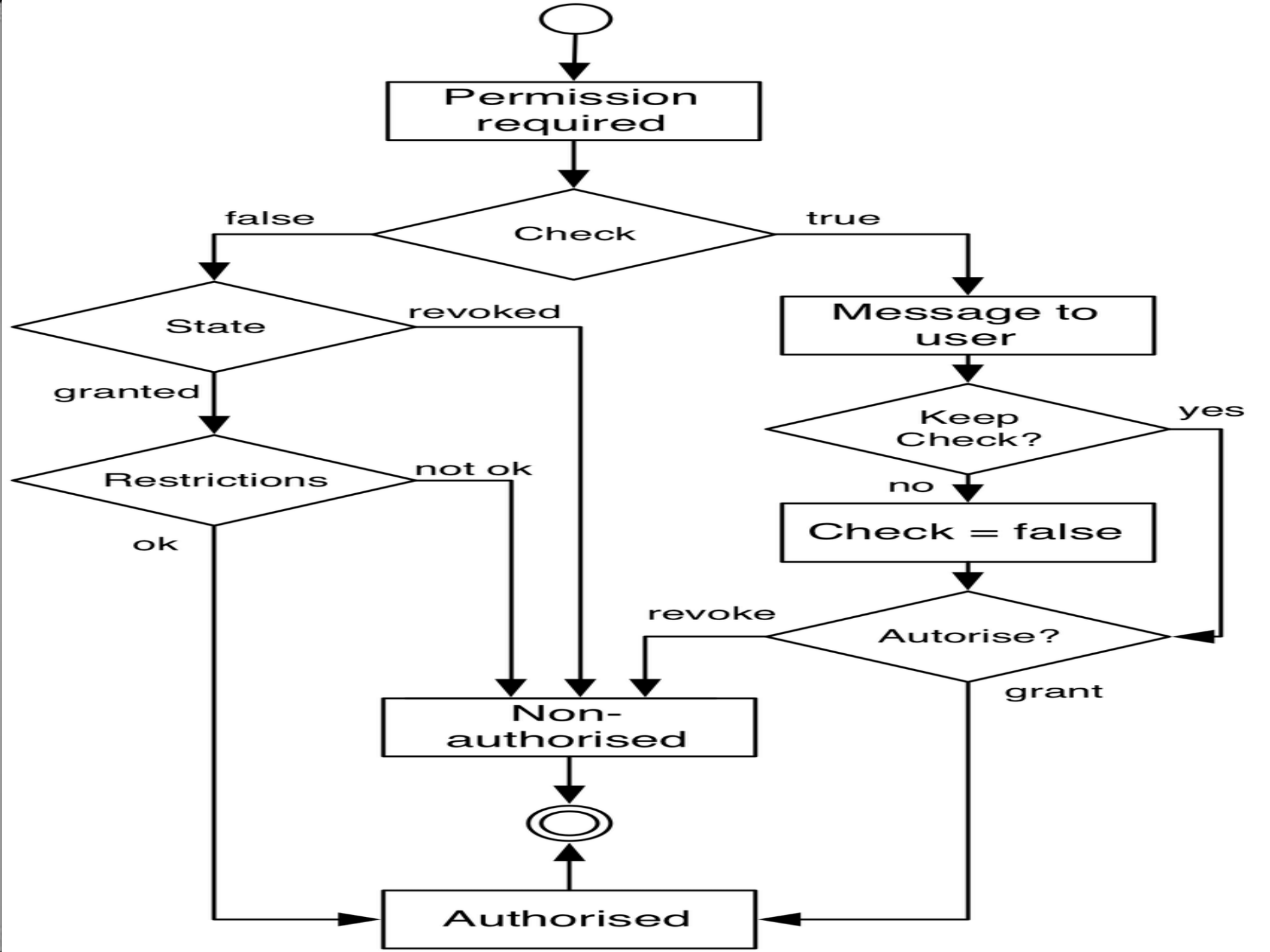
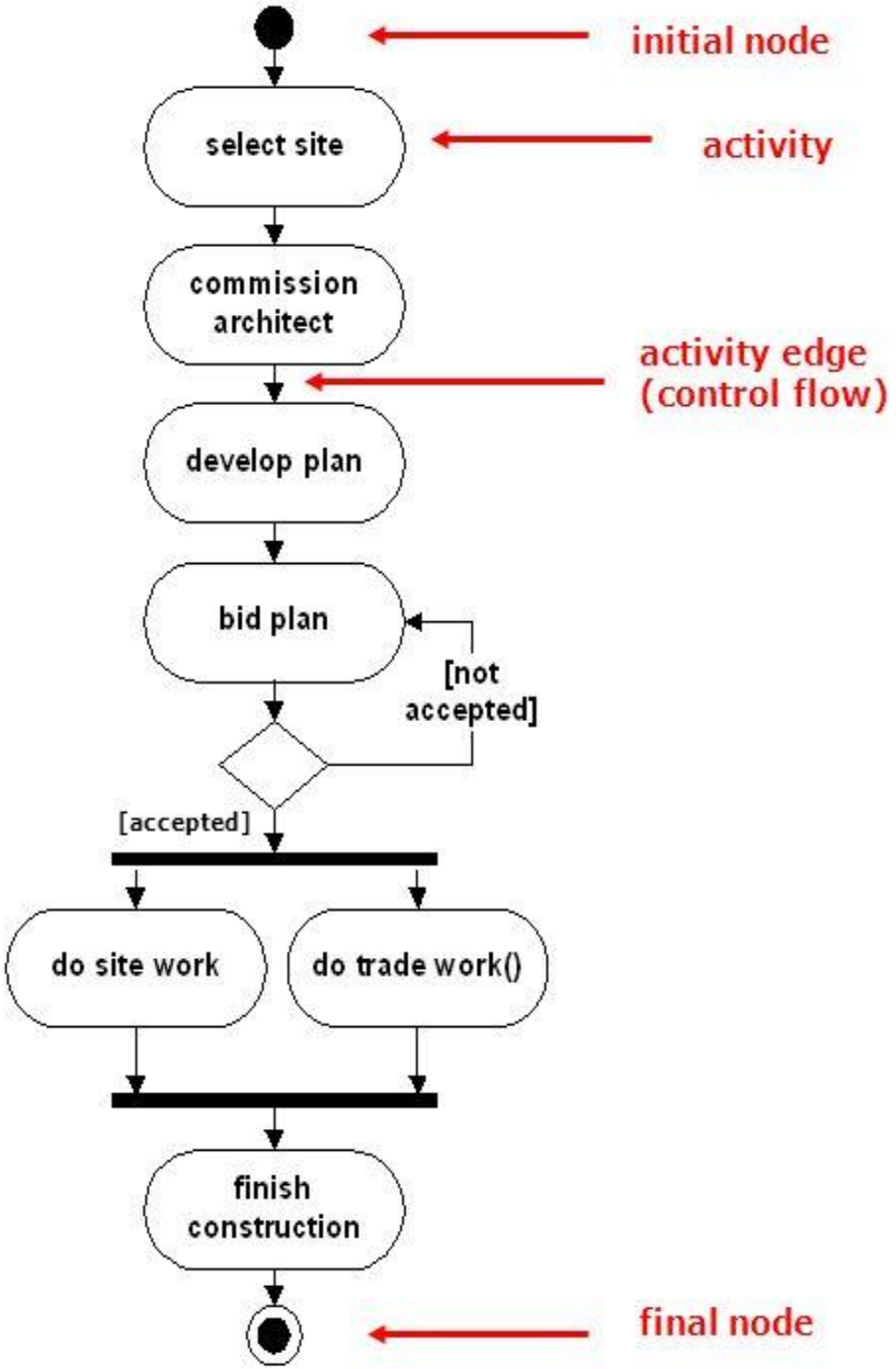


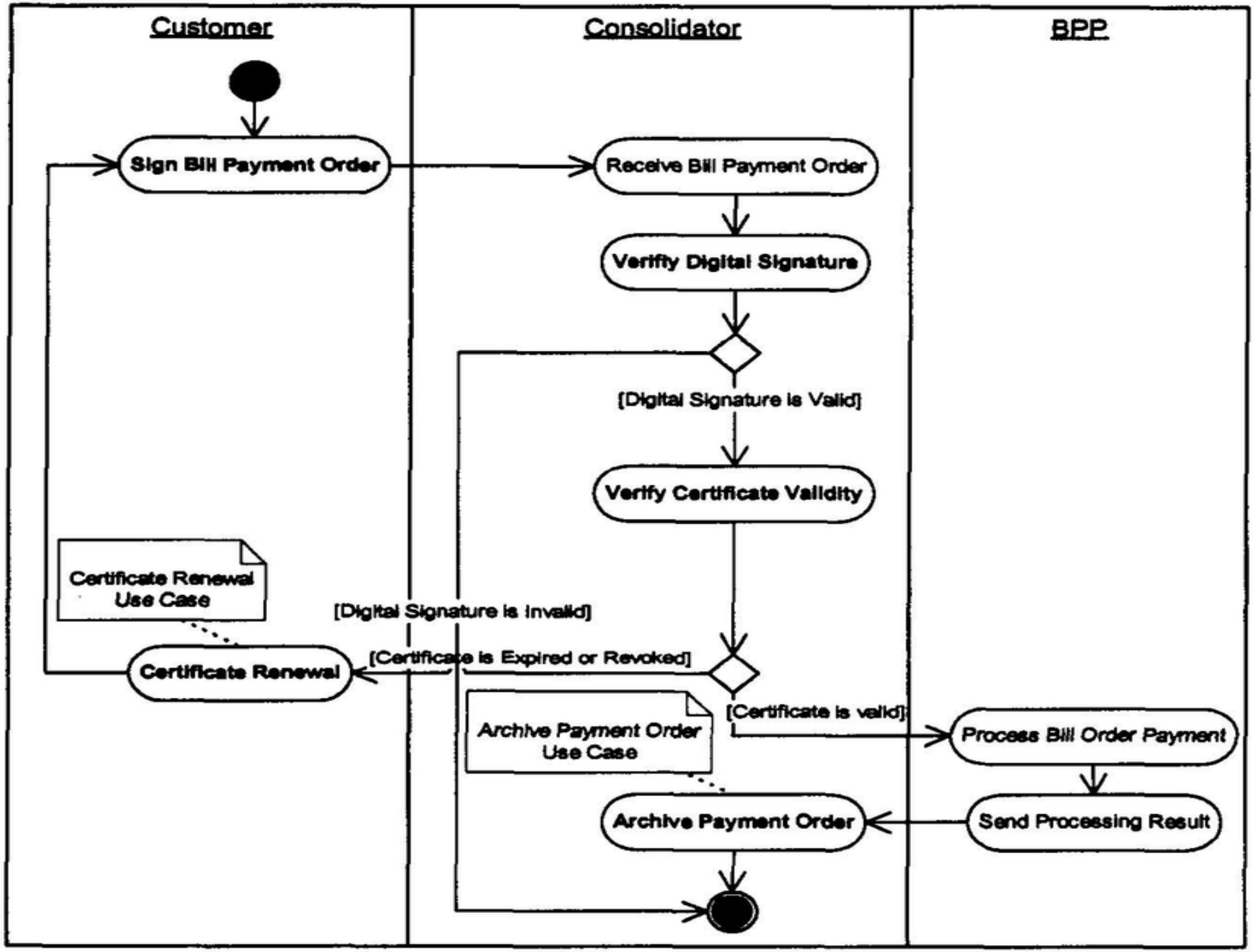
Figure from IBM


Permission required
Check
false
true
State
revoked
granted
Restrictions
not ok
ok
Message to user
Keep Check?
yes
no
Check = false
Autorise?
revoke
grant
Non-authorised
Authorised

activity diagram

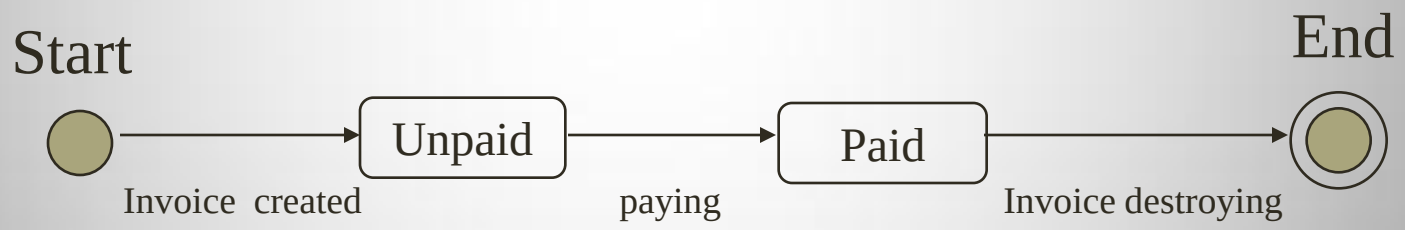


construction system (not automated)


Customer
Consolidator
BPP
Sign Bill Payment Order
Receive Bill Payment Order
Verify Digital Signature
[Digital Signature is Valid]
Verify Certificate Validity
Certificate Renewal Use Case
[Digital Signature is Invalid]
[Certificate is Expired or Revoked]
Certificate Renewal
[Certificate is valid]
Archive Payment Order Use Case
Process Bill Order Payment
Archive Payment Order
Send Processing Result

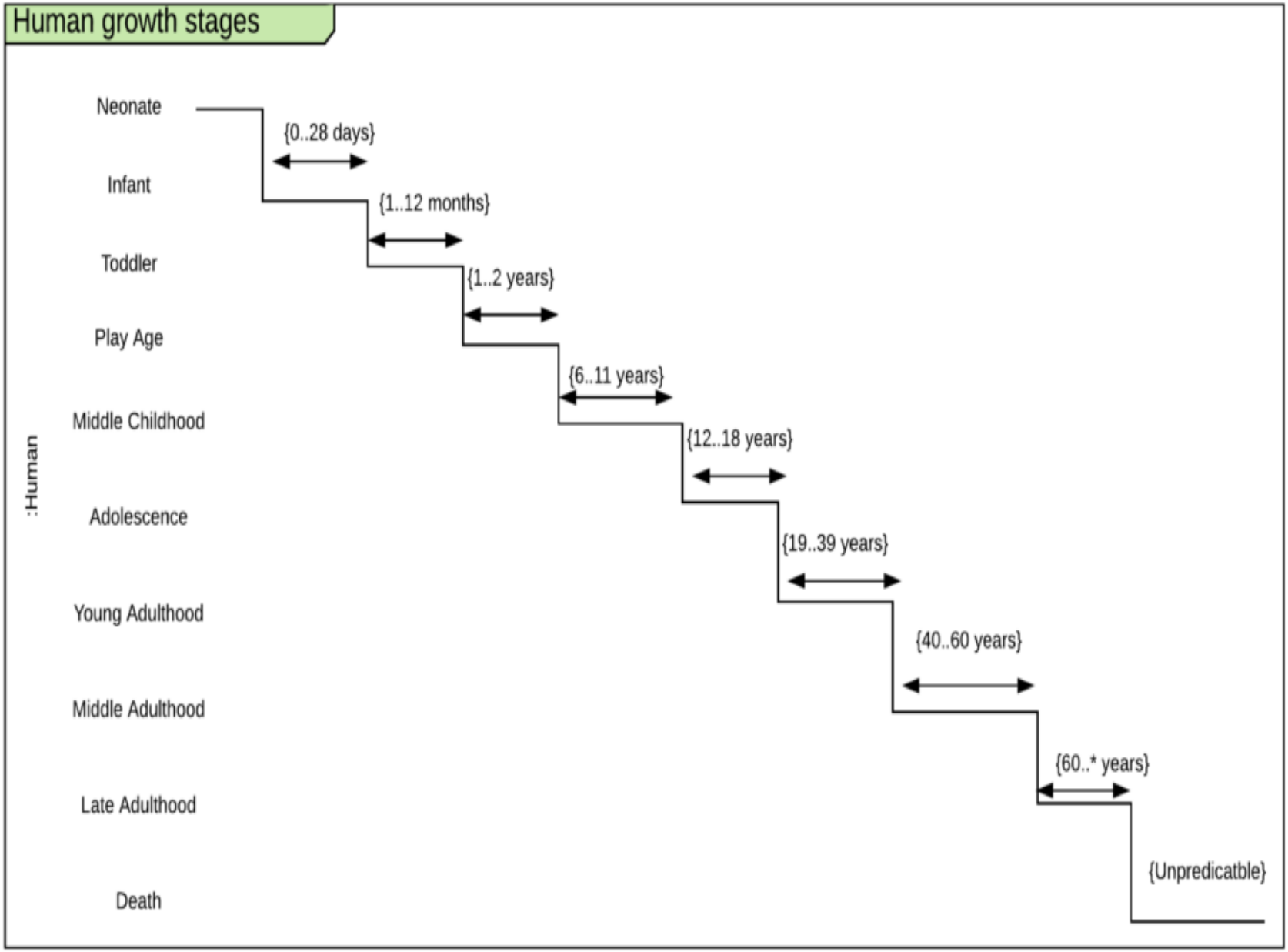
State Diagrams

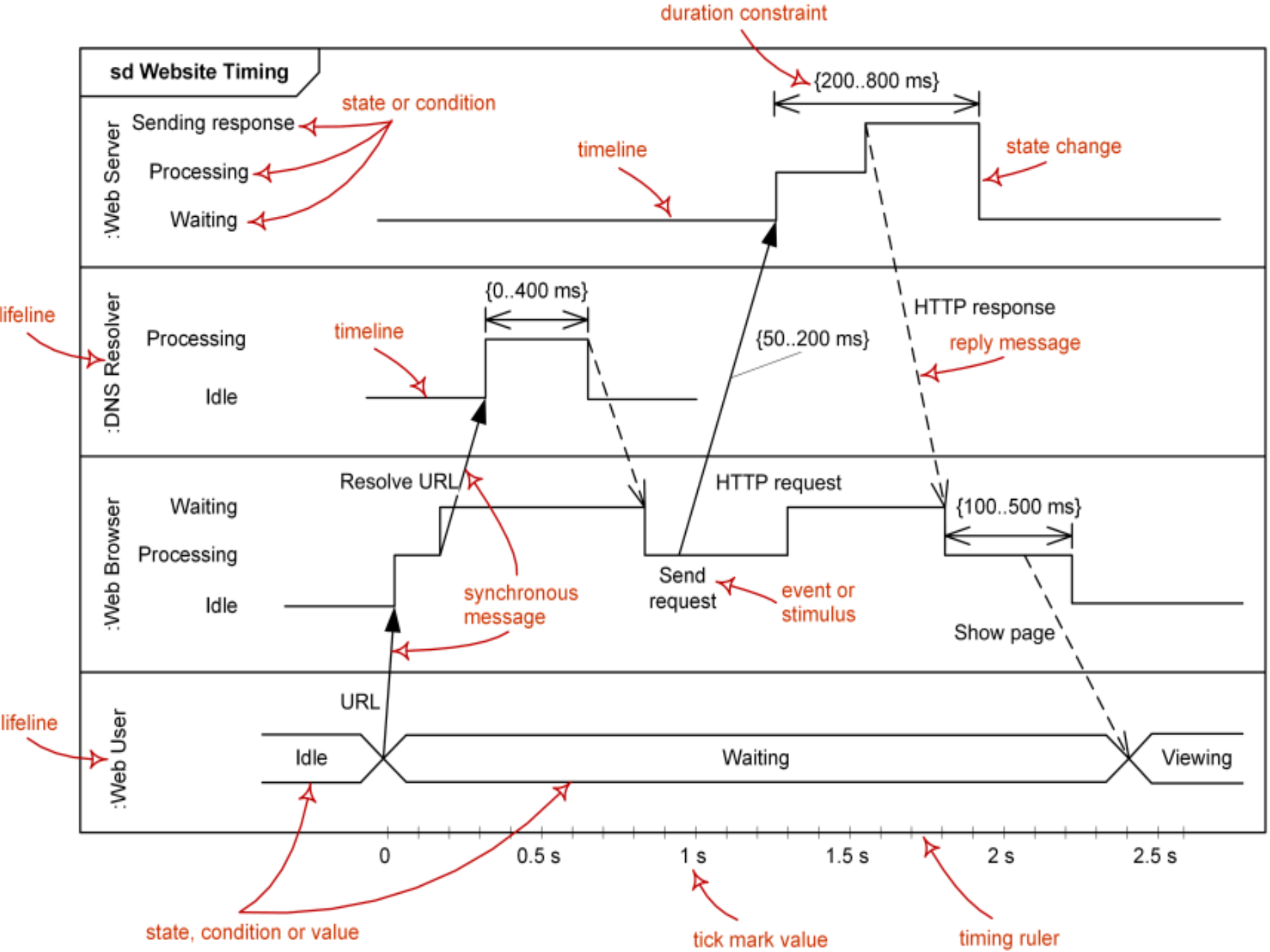
State Diagrams show the sequences of states an object goes through during its life cycle in response to stimuli, together with its responses and actions; an abstraction of all possible behaviors.

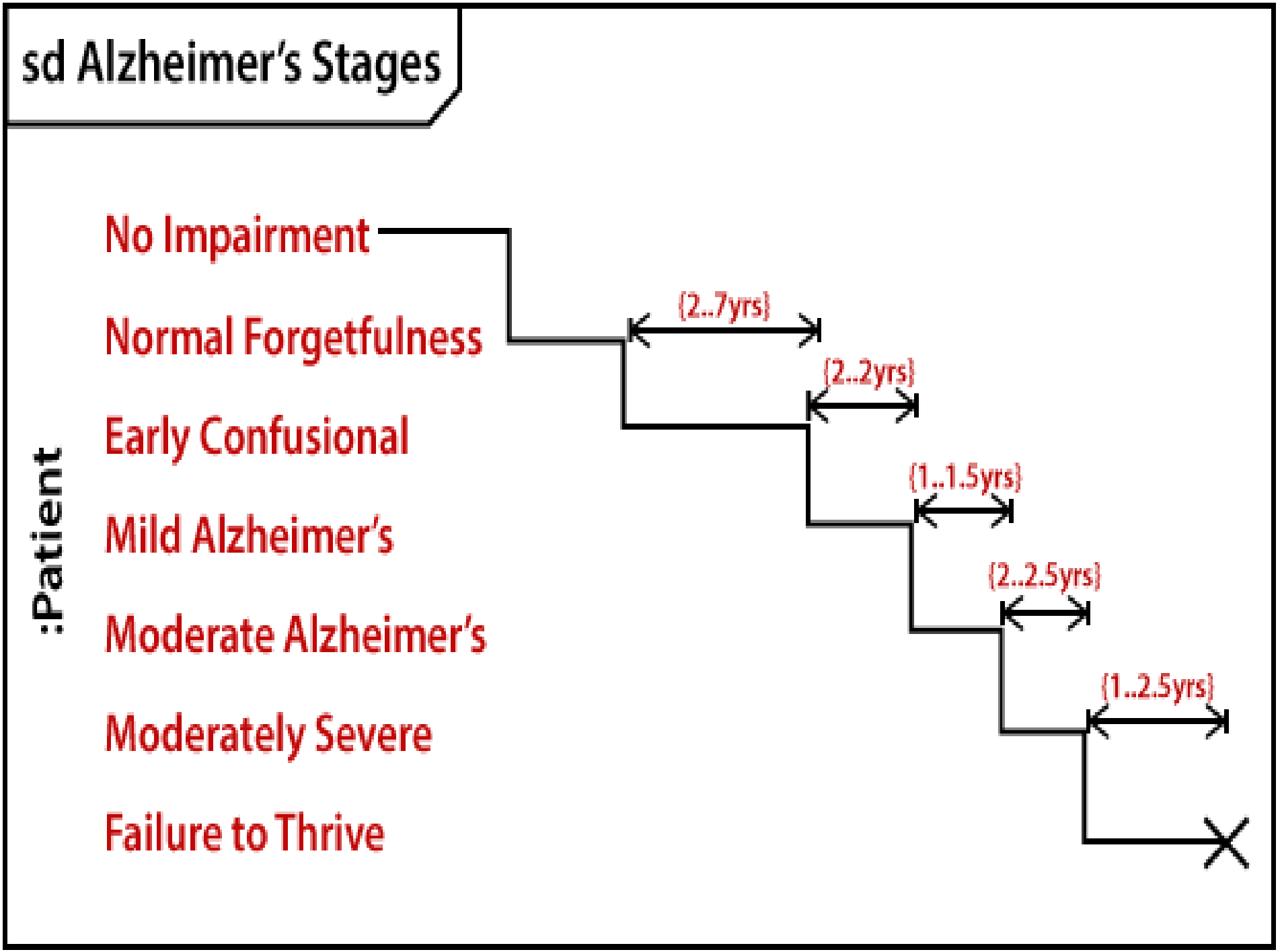


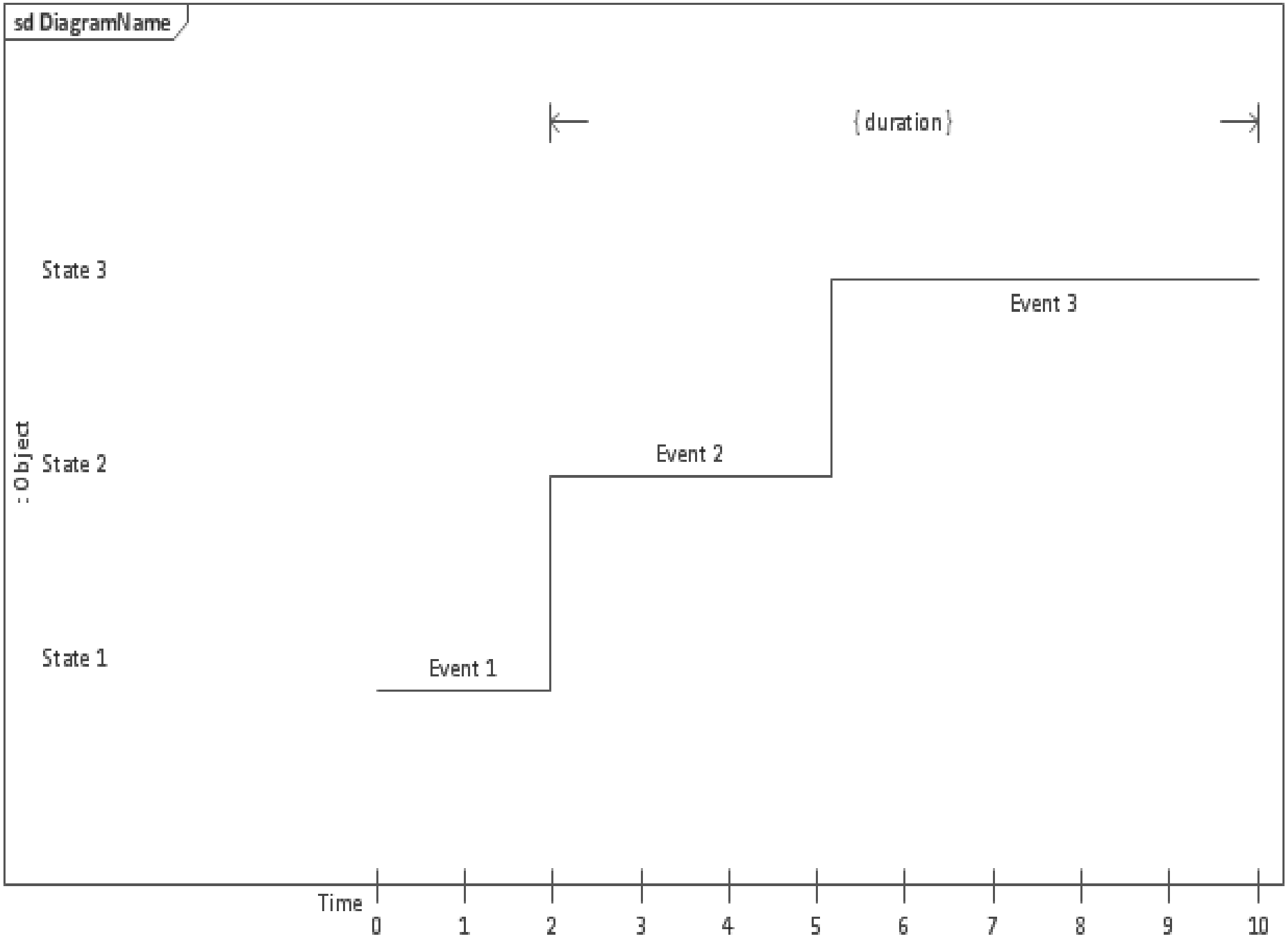
Time Diagram

- show interactions between primary with **time**
- focus on conditions changing within and among **lifelines**
- describe behavior of individual **classifiers** and interactions of classifiers,
- focusing attention on time of events


Human growth stages
:Human
Neonate
{0..28 days}
Infant
{1..12 months}
Toddler
{1..2 years}
Play Age
{6..11 years}
Middle Childhood
{12..18 years}
Adolescence
{19..39 years}
Young Adulthood
{40..60 years}
Middle Adulthood
{60..* years}
Late Adulthood
{Unpredicatble}
Death


sd Website Timing
duration constraint
{200..800 ms}
state or condition
:Web Server
Sending response
Processing
Waiting
timeline
state change
lifeline
:DNS Resolver
Processing
Idle
{0..400 ms}
timeline
{50..200 ms}
HTTP response
reply message
:Web Browser
Waiting
Processing
Idle
Resolve URL
HTTP request
{100..500 ms}
synchronous message
Send request
event or stimulus
Show page
lifeline
:Web User
URL
Idle
Waiting
Viewing
0
0.5 s
1 s
1.5 s
2 s
2.5 s
state, condition or value
tick mark value
timing ruler

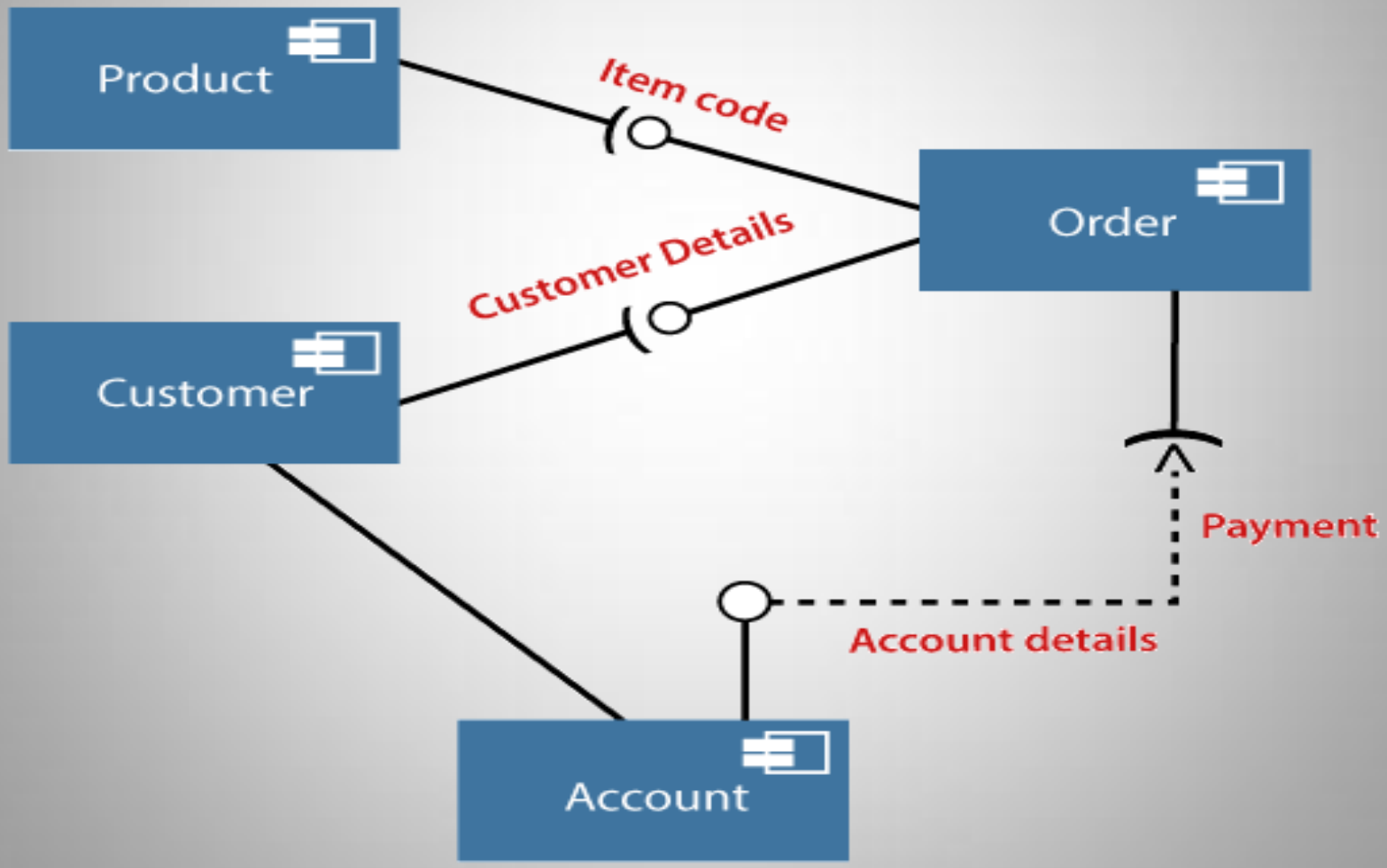

sd Alzheimer's Stages
:Patient
No Impairment
Normal Forgetfulness
Early Confusional
Mild Alzheimer's
Moderate Alzheimer's
Moderately Severe
Failure to Thrive
{2..7yrs}
{2..2yrs}
{1..1.5yrs}
{2..2.5yrs}
{1..2.5yrs}

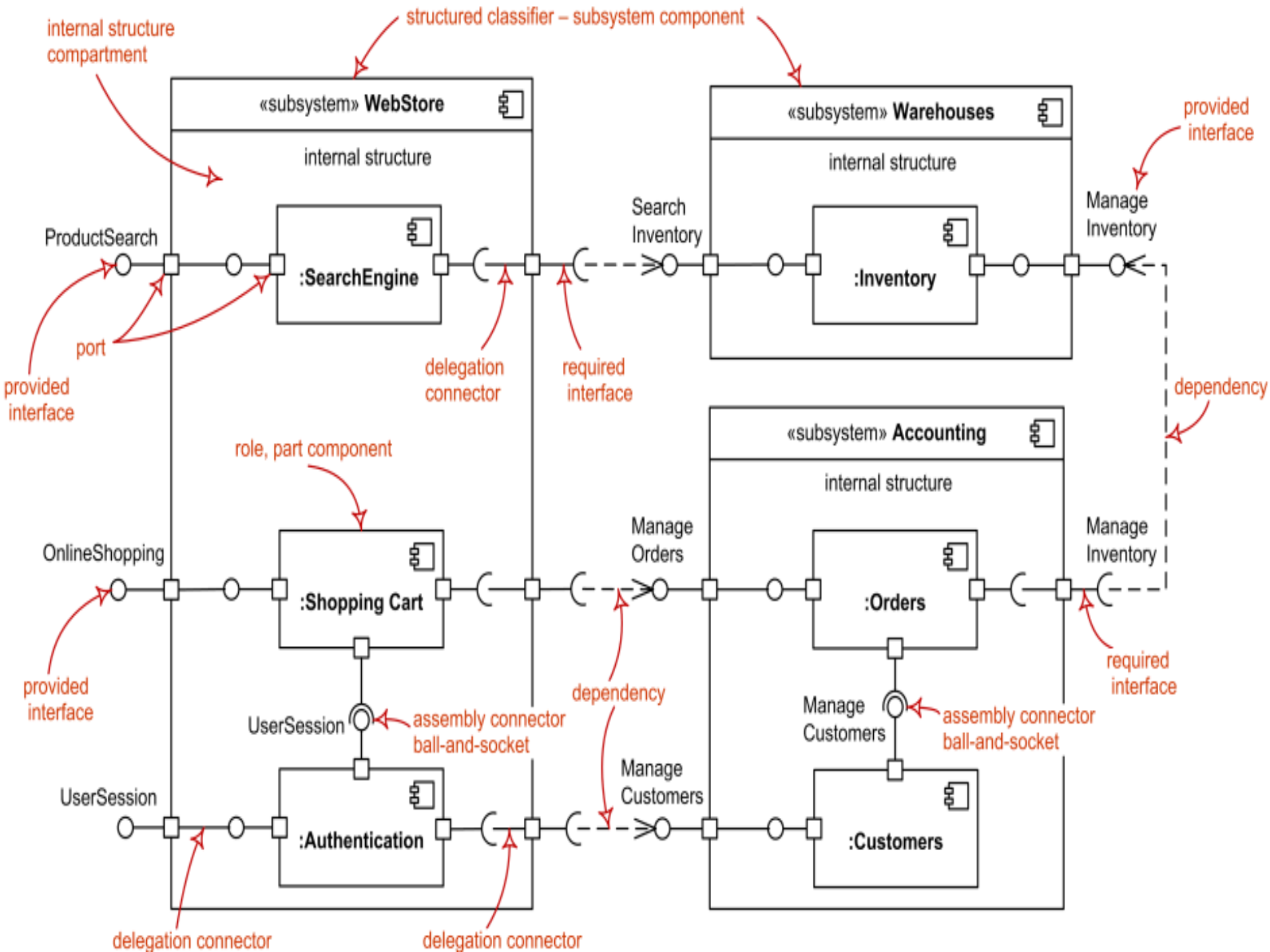

sd DiagramName
{duration}
State 3
Event 3
: Object
State 2
Event 2
State 1
Event 1
Time
0
1
2
3
4
5
6
7
8
9
10

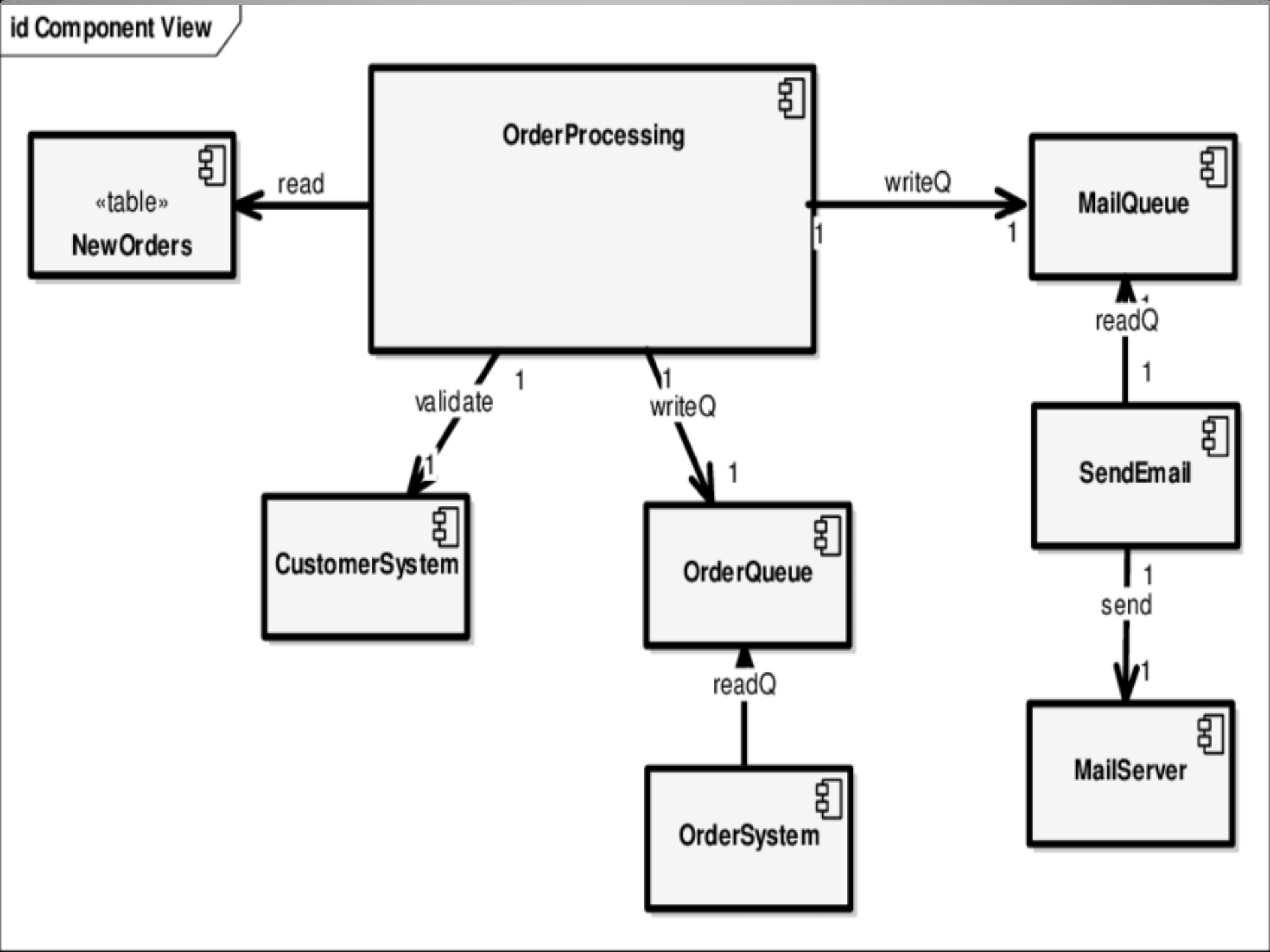
Component Diagram

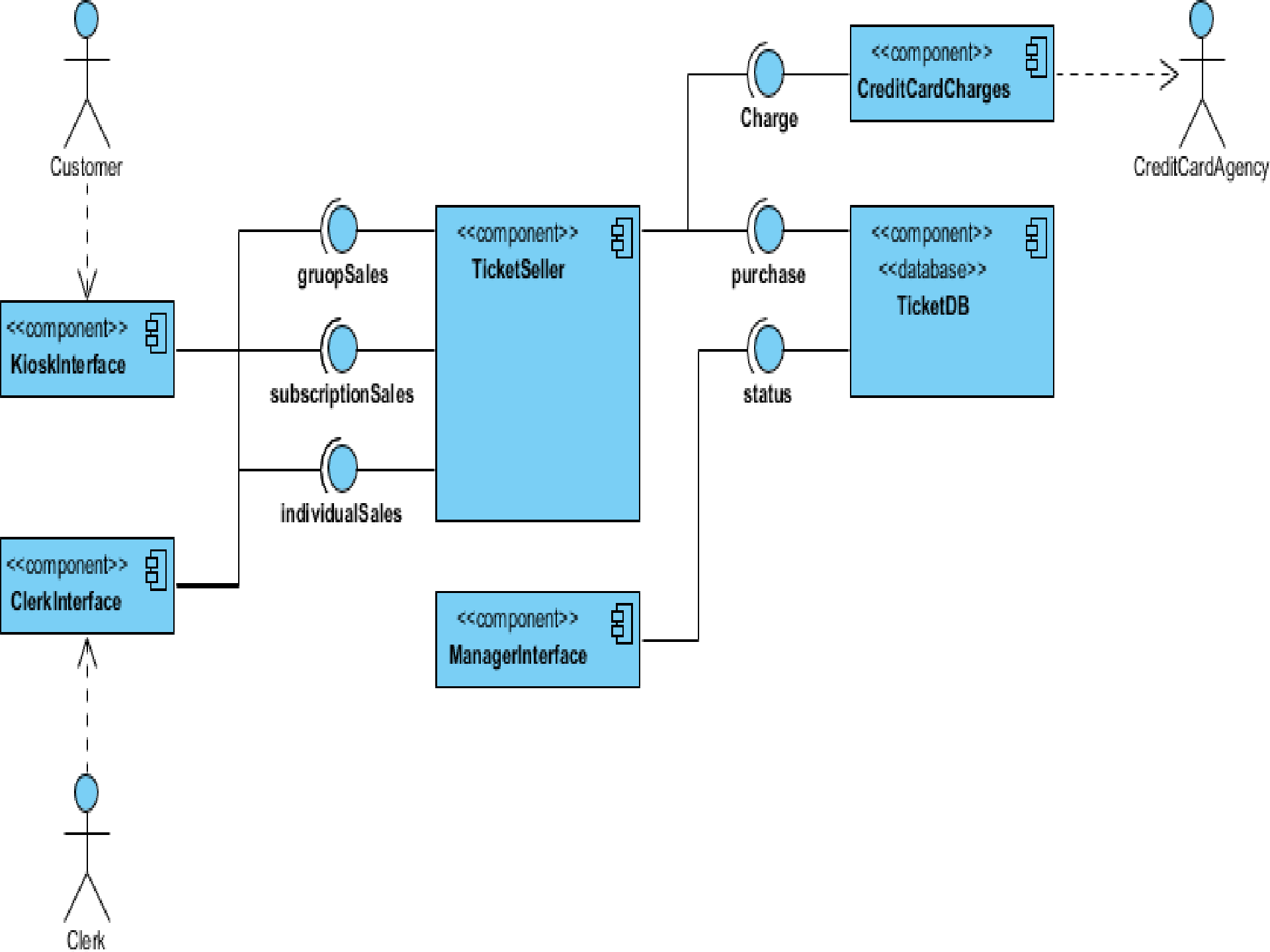
- used to break down a large object-oriented system into the smaller components
- models the physical view of a system such as executables, files, libraries, etc.
- visualizes the relationships as well as the organization between the components present in the system
- A component is a single unit of the system, which is replaceable and executable

component diagram for an online shopping system



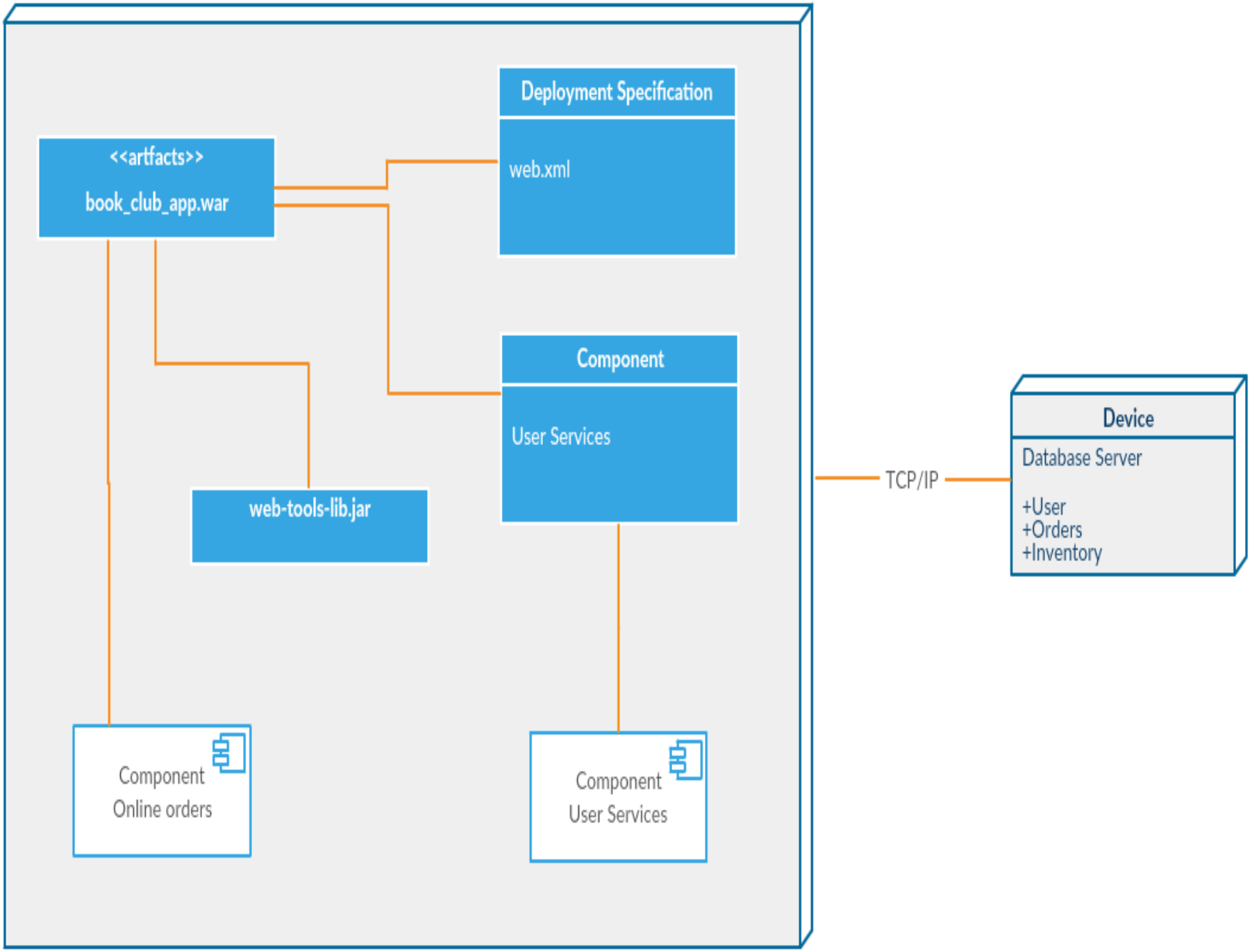

internal structure compartment
structured classifier – subsystem component
«subsystem» WebStore
internal structure
«subsystem» Warehouses
internal structure
provided interface
ProductSearch
:SearchEngine
Search Inventory
:Inventory
Manage Inventory
port
delegation connector
required interface
provided interface
dependency
«subsystem» Accounting
internal structure
role, part component
OnlineShopping
:Shopping Cart
Manage Orders
:Orders
Manage Inventory
provided interface
dependency
required interface
UserSession
assembly connector ball-and-socket
Manage Customers
assembly connector ball-and-socket
UserSession
:Authentication
Manage Customers
:Customers
delegation connector
delegation connector

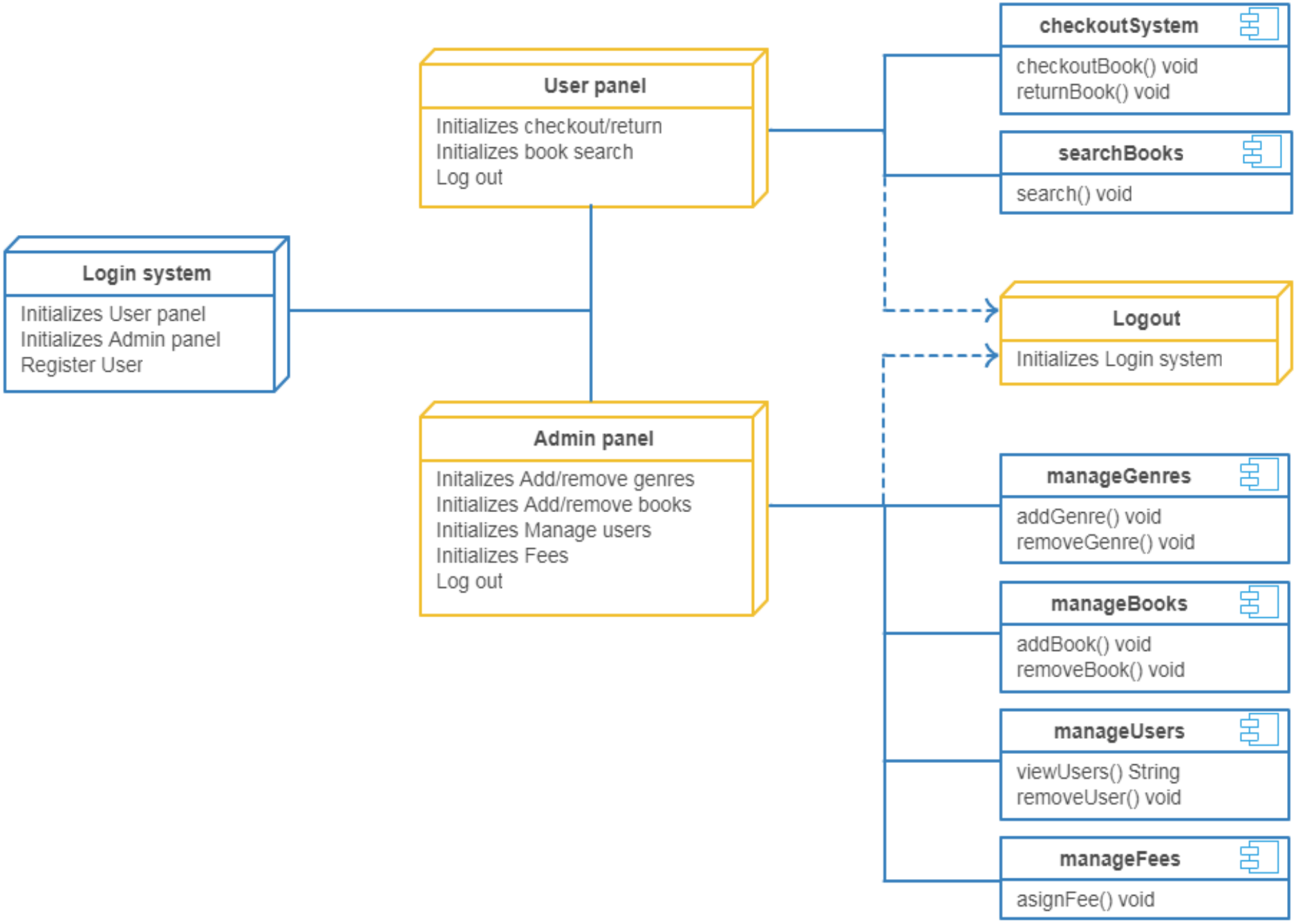

id Component View
OrderProcessing
«table»
NewOrders
read
writeQ
1
1
MailQueue
readQ
1
1
SendEmail
1
send
1
MailServer
validate
1
1
1
writeQ
1
CustomerSystem
OrderQueue
readQ
OrderSystem

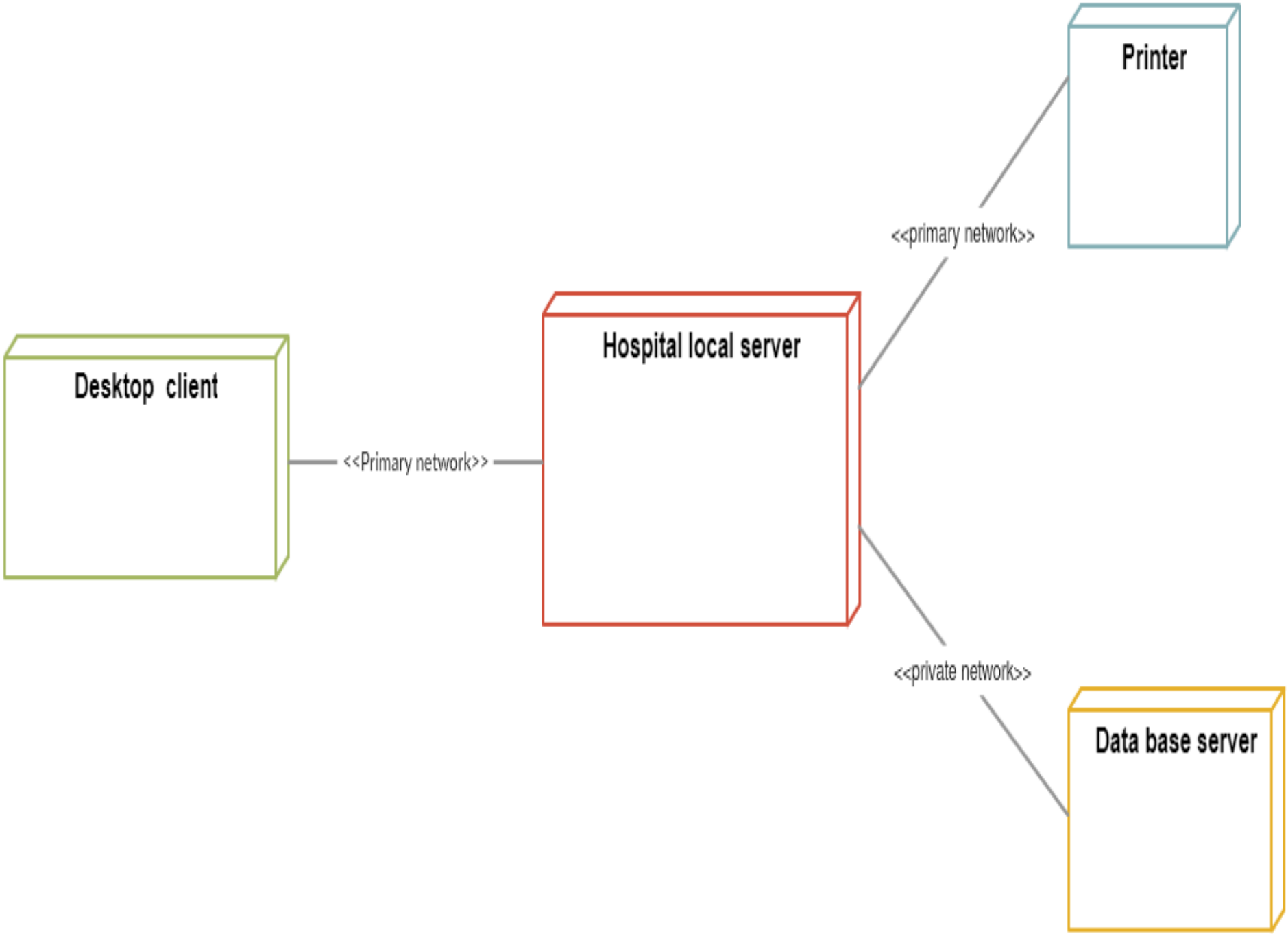

Customer
<<component>>
KioskInterface
<<component>>
ClerkInterface
Clerk
gruopSales
subscriptionSales
individualSales
<<component>>
TicketSeller
<<component>>
ManagerInterface
Charge
purchase
status
<<component>>
CreditCardCharges
<<component>>
<<database>>
TicketDB
CreditCardAgency

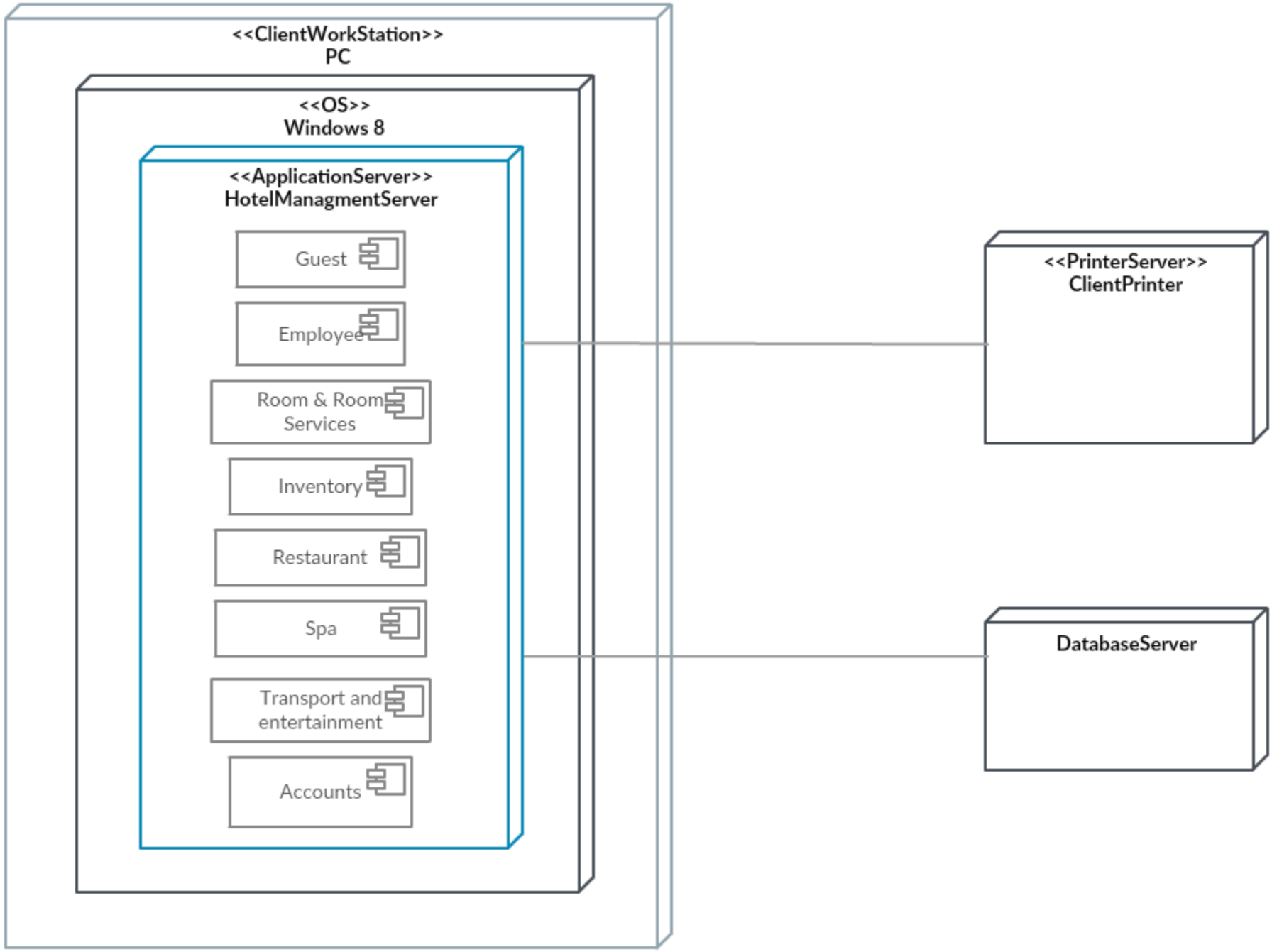
Deployment Diagram

- shows the execution architecture of a system
- visualize the physical hardware and software of a system
- model the hardware topology of a system (logical components of a system)


Deployment Specification
web.xml
<<artfacts>>
book_club_app.war
Component
User Services
web-tools-lib.jar
TCP/IP
Device
Database Server
+User
+Orders
+Inventory
Component
Online orders
Component
User Services


checkoutSystem
checkoutBook() void
returnBook() void
User panel
Initializes checkout/return
Initializes book search
Log out
searchBooks
search() void
Login system
Initializes User panel
Initializes Admin panel
Register User
Logout
Initializes Login system
Admin panel
Initalizes Add/remove genres
Initializes Add/remove books
Initializes Manage users
Initializes Fees
Log out
manageGenres
addGenre() void
removeGenre() void
manageBooks
addBook() void
removeBook() void
manageUsers
viewUsers() String
removeUser() void
manageFees
asignFee() void


Printer
<<primary network>>
Hospital local server
Desktop client
<<Primary network>>
<<private network>>
Data base server


<<ClientWorkStation>>
PC
<<OS>>
Windows 8
<<ApplicationServer>>
HotelManagmentServer
Guest
Employee
Room & Room Services
Inventory
Restaurant
Spa
Transport and entertainment
Accounts
<<PrinterServer>>
ClientPrinter
DatabaseServer